In the Name of God



Iran University of Science and Technology

School of Electrical Engineering

Department of Control Engineering

M.SC. SEMINAR REPORT

A Review of Path Planning Methods for Swarm Robots

By

Arshia Goshtasbi

Supervisor

Dr. Mohammad Farrokhi

December 2025

Extended edition, November 2026

Abstract

Swarm robotics, inspired by nature, deals with the decentralized coordination of large groups of robots, and path planning is one of its fundamental challenges. This seminar reviews and analyzes the various path planning methods for swarm robots, ranging from bio-inspired algorithms (such as ant colony optimization and particle swarm optimization) and sampling-based methods (such as RRT and PRM) to novel approaches based on reinforcement learning.

The analysis shows that classical algorithms face computational limitations at large scales, whereas decentralized and bio-inspired methods offer better scalability and flexibility in dynamic environments. It is concluded that the future of this field depends on the development of hybrid and learning-based algorithms that can strike a balance between optimality, efficiency, and real-time adaptability.

Keywords: Swarm robotics; path planning; swarm intelligence; decentralized control; bio-inspired algorithms.

Contents

Abstract i

List of Figures v

List of Tables vii

List of Abbreviations viii

1 Introduction 1

1.1 Definition and Characteristics of Swarm Robotics . . . 1
1.1.1 Definition of Swarm Robotics . . . 1
1.1.2 Inspiration from Nature . . . 2
1.1.3 Key Characteristics of Swarm Robotic Systems . . . 3
1.2 Distinguishing Swarm Robotics from Other Paradigms . . . 4
1.3 Importance of Path Planning in Swarm Robotics . . . 4
1.3.1 The Fundamental Challenge in Multi-Robot Systems . . . 5
1.3.2 Specific Challenges of Swarm Path Planning . . . 6
1.4 Objectives and Structure of the Report . . . 7
1.4.1 Objectives . . . 7
1.4.2 Structure . . . 8

2 Fundamental Concepts and Methodologies in Swarm Control 9

2.1 Swarm Intelligence and Self-Organization . . . 9
2.1.1 Definition and Concepts of Swarm Intelligence . . . 9
2.1.2 Principles of Self-Organization and Stigmergy . . . 10
2.2 Control Architectures for Swarm Robotics . . . 12
2.2.1 Decentralized Control . . . 12
2.2.2 Hierarchical Control Architectures . . . 14
2.2.3 Behavior-Based Control Methods . . . 15
2.2.4 Hybrid Control Methods . . . 16
2.2.5 Virtual Structure and Virtual Linkage Approaches . . . 17
2.3 Communication and Sensing in Swarms . . . 18
2.3.1 Local Interactions and Limited Communication . . . 18
2.3.2 Communication Protocols and Network Topology . . . 20
2.3.3 Sensor Models and Data Acquisition . . . 21
2.4 Basic Concepts of Path Planning . . . 22
2.4.1 Definition of Path Planning . . . 22
2.4.2 Global versus Local Planning . . . 23
2.4.3 Objectives and Metrics . . . 24

3 Path Planning Algorithms for Swarm Robots 26

3.1 Bio-Inspired Algorithms . . . 26
3.1.1 Ant Colony Optimization (ACO) . . . 26
3.1.2 Particle Swarm Optimization (PSO) . . . 29
3.1.3 Bee-Inspired Algorithms . . . 30
3.1.4 Other Swarm Intelligence Algorithms . . . 32
3.2 Graph-Based and Sampling-Based Methods . . . 33
3.2.1 Probabilistic Roadmap (PRM) . . . 33
3.2.2 Rapidly-Exploring Random Trees (RRT) . . . 34
3.2.3 The A* Algorithm and Grid-Based Methods . . . 36
3.3 Potential Field Methods . . . 38
3.3.1 Artificial Potential Fields (APF) . . . 38
3.4 Reinforcement Learning and Deep Learning Approaches . . . 40
3.4.1 Reinforcement Learning (RL) . . . 40
3.4.2 Deep Reinforcement Learning (DRL) . . . 42
3.4.3 Imitation Learning (IL) . . . 43
3.5 Hybrid Approaches . . . 44
3.5.1 Integrating Multiple Path Planning Strategies . . . 44
3.5.2 Other Hybrid Patterns and Evidence . . . 45
3.6 Virtual Tube-Based Planning . . . 46
3.6.1 The Concept of Virtual Tubes . . . 46
3.6.2 Optimal Virtual Tube Planning . . . 47
3.7 Summary . . . 48

4 Challenges and Applications of Path Planning in Swarm Robots 50

4.1 Challenges in Dynamic and Complex Environments . . . 50
4.1.1 Obstacle Avoidance and Collision Resolution . . . 50
4.1.2 Unknown and Dynamic Environments . . . 51
4.1.3 Adaptation and Real-Time Replanning . . . 53
4.2 Scalability and Robustness . . . 54
4.2.1 Scalability in Swarm Systems . . . 54
4.2.2 Robustness and Fault Tolerance . . . 55
4.3 Task Allocation and Coordination . . . 56
4.3.1 Dynamic Task Allocation . . . 56
4.3.2 Cooperative Control and Manipulation . . . 57
4.4 Real-World Applications of Swarm Robotics . . . 59
4.4.1 Search and Rescue . . . 59
4.4.2 Environmental Monitoring and Mapping . . . 60
4.4.3 Agriculture . . . 61
4.4.4 Space Exploration and Construction . . . 61
4.4.5 Medical Applications . . . 62
4.4.6 Object Transportation and Manipulation . . . 62
4.4.7 Security and Defense . . . 62
4.4.8 Logistics . . . 62
4.4.9 Patrolling and Surveillance . . . 63
4.4.10 Data Analysis and Graph Partitioning . . . 63

5 Summary, Conclusions, and Future Work 64

5.1 Summary of Path Planning Approaches 64
5.1.1 Overview of Deterministic, AI-Based, and Hybrid Approaches 64
5.1.2 Methodologies and Their Principles 64
5.2 Conclusion 65
5.2.1 Advantages of Swarm Robotics 66
5.2.2 The Critical Role of Advanced Path Planning 66
5.3 Future Research Directions 67
5.3.1 Advanced Control and Learning Strategies 67
5.3.2 Human–Swarm Interaction 67
5.3.3 Hardware and Simulation 68
5.3.4 Ethics and Societal Impact 68
5.3.5 Dynamic Formation Generation and Route Planning 68
5.3.6 Sensing-Aware Planning 69
5.3.7 Multi-Objective Planning and Constraint Handling 69

References 70

List of Figures

1.1 The position of swarm robotics within mobile and multi-robot systems. 1
1.2 Classification of path planning methods (redrawn after [15]). 5
1.3 Control architectures in multi-agent robotics: (a) centralized, (b) decentralized, and (c) distributed. Processing units are shown in red. 5

2.1 Pitchfork bifurcation of the trail model with $\alpha = 2$: (a) equilibria of the flux on bridge 1 as a function of the incoming flux φ, with numerical integration as dots; (b) the asymmetry ΔC, which becomes positive above $\varphi_c = 2$. 12
2.2 A mechanical linkage and its virtual counterpart, the virtual linkage (redrawn after [43]).. 18

3.1 A simulated double-bridge experiment: share of ants on the short branch for 20 colonies (thin lines) and their mean (thick line). Choices follow Equation (2.1) with $k = 20$, $\alpha = 2$, and the long branch is twice as long, so it receives half the pheromone per crossing. 27
3.2 Using ACO to tune the parameters of an NLPID controller (redrawn after [55]). 28
3.3 PSO path planning: each particle encodes four waypoints between the start and the goal, and the cost is the path length plus a penalty for entering an inflated obstacle. (a) The best waypoints after 150 iterations, smoothed with a cubic B-spline; the grey dots are the final positions of the swarm. (b) Convergence of the best cost. 31
3.4 PRM in a cluttered workspace: collision-free milestones are connected to their ten nearest neighbours, and Dijkstra's algorithm finds the path on the roadmap. 34
3.5 RRT and RRT-Connect in the workspace of Figure 3.4, with the same step size and random seed. RRT-Connect grows a tree from each end and joins them with its CONNECT step, so it needs fewer iterations. 35
3.6 Dijkstra's algorithm and A* on the same 8-connected grid. Both return the same optimal path, but the admissible octile heuristic lets A* expand far fewer cells. 37
3.7 An approximate generalized Voronoi diagram, built from the ordinary Voronoi diagram of points sampled on the obstacle boundaries. The path follows the roadmap through the narrow door and keeps the largest possible distance from the walls. 37
3.8 The force-field view of robot path planning: the arrows show the direction of $F = -\nabla U$. (a) The target attracts and the obstacles repel the robot, which reaches the goal. (b) A U-shaped obstacle creates a local minimum in which the same controller gets stuck. 39

3.9 The CTDE scheme: a centralized critic is trained on the observations and actions of all agents, while each agent executes its own decentralized actor. . . 41
3.10 The imitation learning approach: expert demonstrations form a training dataset from which a policy $\pi_\theta(a_t \mid o_t)$ is learned by supervised learning. 43
3.11 Block diagram of the FNN algorithm combined with A* (redrawn after the description in [37]).. 45
3.12 A linear virtual tube with convex-hull terminals. Only the four vertex trajectories (blue) are planned; each robot starts at a convex combination of the vertices of C_0 and follows the same combination of the vertex trajectories (red). 46

4.1 Leader–follower formation tracking: five followers start from random positions and converge, through local consensus on their formation errors, to a wedge around the leader (markers every few seconds).. 59

List of Tables

1.1 Comparison of swarm robotics with other paradigms [2]. 4

4.1 Examples of real-world applications of swarm robotics. 60

List of Abbreviations

ABC	Artificial Bee Colony
ACO	Ant Colony Optimization
ACS	Ant Colony System
AI	Artificial Intelligence
APF	Artificial Potential Field
AS	Ant System
AUV	Autonomous Underwater Vehicle
BA	Bees Algorithm
BBC	Behavior-Based Control
BVC	Buffered Voronoi Cell
CRLB	Cramér–Rao Lower Bound
CTDE	Centralized Training with Decentralized Execution
DDPG	Deep Deterministic Policy Gradient
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
DRRT	Dynamic Rapidly-exploring Random Tree
DTSP	Distributed Task Selection Problem
DWA	Dynamic Window Approach
EP	Evolutionary Programming
ERRT	Execution-extended Rapidly-exploring Random Tree
FNN	Fuzzy Neural Network
FSM	Finite State Machine
FWA	Fireworks Algorithm
GA	Genetic Algorithm
GAP_SAC	Gated-Attention Prioritized-replay Soft Actor-Critic
GFWA	Guided Fireworks Algorithm
GVD	Generalized Voronoi Diagram
GWO	Grey Wolf Optimizer
HASP	Hierarchical Action Selection Process
HBMO	Honey Bee Mating Optimization
HH-ABC	Hierarchical–Hybrid Artificial Bee Colony
IL	Imitation Learning
IMU	Inertial Measurement Unit
IPF	Improved Potential Field
IPSO-DAI	Integral Particle Swarm Optimization with Dispersed Accelerator Information
IR	Infrared
MANET	Mobile Ad Hoc Network
MAPRM	Medial-Axis Probabilistic Roadmap
MARL	Multi-Agent Reinforcement Learning
MOPSO	Multi-Objective Particle Swarm Optimization

MPC	Model Predictive Control
MPRRT	Multipartite Rapidly-exploring Random Tree
MRTA	Multi-Robot Task Allocation
MTT	Moving Target Tracking
NisB	Nearer is Better (PSO variant)
NLPID	Nonlinear Proportional–Integral–Derivative (controller)
ORCA	Optimal Reciprocal Collision Avoidance
OVTP	Optimal Virtual Tube Planning
PDE	Partial Differential Equation
PFSM	Probabilistic Finite State Machine
PRM	Probabilistic Roadmap
PSO	Particle Swarm Optimization
RF	Radio Frequency
RL	Reinforcement Learning
RRT	Rapidly-exploring Random Tree
RVO	Reciprocal Velocity Obstacles
SAC	Soft Actor-Critic
SaR	Search and Rescue
SCA	Sine Cosine Algorithm
SI	Swarm Intelligence
SLAM	Simultaneous Localization and Mapping
SOMA	Self-Organizing Migrating Algorithm
TD	Temporal Difference
TD3	Twin Delayed Deep Deterministic Policy Gradient
TDOA	Time Difference of Arrival
TLBO	Teaching–Learning-Based Optimization
TSP	Traveling Salesman Problem
UAV	Unmanned Aerial Vehicle
VL	Virtual Linkage
VRP-TW	Vehicle Routing Problem with Time Windows
VS	Virtual Structure
WOA	Whale Optimization Algorithm

CHAPTER 1

Introduction

1.1 Definition and Characteristics of Swarm Robotics

Swarm robotics is an interdisciplinary approach inspired by social insects (such as ants, bees, and termites) and by groups of vertebrates (such as bird flocks and fish schools), in which a large number of simple robots achieve complex group behaviors through local interactions. Swarm robotics is concerned with engineering large-scale collective behaviors that, by exploiting the self-organization of simple agents, yield robust, scalable, and flexible systems [1], [2]. It has become the most common way of designing collective behaviors for groups of robots [3]. As shown in Figure 1.1, swarm robotics is a subset of multi-robot systems.

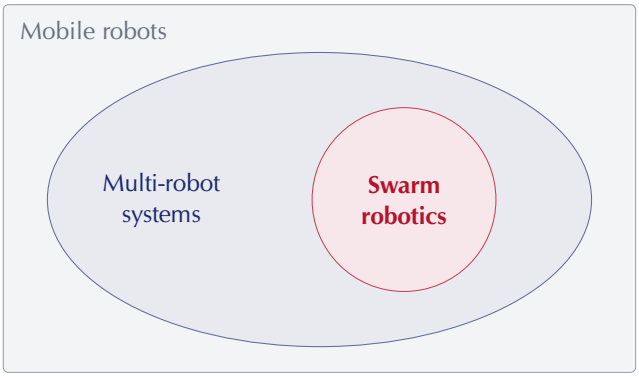


Figure 1.1. The position of swarm robotics within mobile and multi-robot systems.

1.1.1 Definition of Swarm Robotics

Swarm robotics is a way of coordinating multi-robot systems that consist of large populations of relatively simple, physically embodied robots. Its central design problem is to choose the behavior of the individual robots so that a desired macroscopic behavior emerges from local robot–robot and robot–environment interactions, without centralized control [3], [4]. The design approach focuses on scalability, flexibility, and robustness, and it relies on local sensing and communication combined with distributed autonomy [1], [2].

1.1.1.1 Study of Collective Intelligence in Robot Groups

Swarm robotics studies how a group of simple robots, relying on emergent collective intelligence, can cooperatively accomplish tasks that are beyond the capabilities of any single robot [2], [4]:

- **Emergent behavior:** Complex behaviors such as aggregation, dispersion, and collective flight arise spontaneously from simple local rules and give the swarm capabilities that exceed those of the individual.

- **Simple agents:** Swarm robots are typically homogeneous and of low complexity, with minimal on-board storage and computation, and they rely on coordination to achieve complex goals collectively.
- **Minimal control:** The swarm operates with little or no human guidance and achieves decentralized autonomy by following simple cooperative rules; a group that needs continuous human operation is not usually regarded as a swarm [5].

1.1.1.2 Merging Swarm Intelligence with Robotics

Swarm robotics grew out of swarm intelligence and can be read as swarm intelligence applied to embodied robots: it implements the principles of decentralized, self-organized problem solving in robotic teams under a fully distributed control paradigm [1], [6], [7]:

- **The swarm intelligence paradigm:** It employs decentralized, self-organized systems to solve complex problems and provides the computational techniques and inspiration needed for robust collective behavior in robotics.
- **Swarm intelligence in applied robotics:** Models of coordination observed in biological systems are turned into coordination and task-allocation algorithms that can be implemented on robot swarms.
- **Decentralized coordination:** Purely local sensing and communication, together with distributed decisions, eliminate single points of failure and allow the system to grow to large teams while maintaining performance.

1.1.2 Inspiration from Nature

Drawing on nature, and based on observations and mathematical models of biological swarms, this field engineers corresponding coordination and problem-solving mechanisms for robotics. The success of natural systems in solving complex tasks in a decentralized manner, from ant colonies and bee swarms to bird flocks, fish schools and herds of ungulates, provides templates for robust and adaptive robotic collectives [2].

1.1.2.1 Biological Swarms as Models

Biological collectives, from micro-organisms to large vertebrates, show how many simple agents can reach sophisticated collective outcomes, and the behaviors they produce are typically robust, flexible, and scalable [1], [8].

- **Social insects:** Ants, bees, wasps, and termites are the main sources of inspiration. Termites build large and intricately ventilated mounds, army ants organize foraging raids involving up to 200,000 workers, and groups of ants carry prey far heavier than any single worker could move [8], [9].
- **Tasks transferred to robotics:** The behaviors that have been carried over to robots include foraging (collecting scattered items), nest construction, division of labor and task allocation, and path finding; desert ants, for example, solve navigation problems by path integration [2], [10].
- **Vertebrate groups:** Bird flocks, fish schools, and insect swarms provide models of collective motion. Reynolds' *boids* showed that three local rules (separation, alignment, and cohesion) are enough to reproduce realistic flocking [11].

The common lesson of these systems is that collective intelligence emerges through self-organization: individuals that are simple and interact only locally can still reach goals far beyond their own capacities [9].

1.1.2.2 Translating Natural Algorithms to Robotic Control

Turning a natural behavior into a robot controller usually follows three steps [8], [12], [13]:

1. **Modeling and simulation:** The collective behavior is first studied in the natural system, then modeled mathematically, and the model is used as the basis for artificial variants; swarm intelligence treats such natural phenomena as computational processes from which algorithms can be derived.
2. **Emulating collective features:** The resulting algorithms keep the features that make the natural swarm work, such as decentralized decisions, self-organization, and adaptive responses to the environment, and apply them to optimization, control, classification, and routing.
3. **Bio-inspired algorithms:** The best-known outcomes are metaheuristics: ant colony optimization (ACO) mimics pheromone trail laying in foraging ants and is used for routing and coverage, while particle swarm optimization (PSO) mimics the coordinated motion of flocks and schools to improve candidate solutions iteratively. Both are reviewed in Chapter 3.

Compared with planners that rely on an explicit model of the environment, these methods let robots navigate complex environments adaptively and support decentralized decision-making, which is valuable for robustness and distributed problem solving [13].

1.1.3 Key Characteristics of Swarm Robotic Systems

Swarm robotic systems prioritize autonomy, decentralization, scalability, robustness, flexibility, and limited individual capabilities, so that high performance arises from a collective of simple components rather than from a few complex units [1], [3].

1.1.3.1 Self-Organization and Decentralization

Swarm robotics dispenses with centralized control: each robot acts autonomously based on local sensing and interaction with its neighbors, and global behaviors emerge from repeated local exchanges [1].

- **Decentralized control:** There is no leader or central authority, so the vulnerability of a single point of failure disappears.
- **Self-organization:** Complex global patterns form through local interactions among simple robots and adapt to environmental cues and task demands. Since the macroscopic behavior cannot be programmed directly, the swarm engineer designs the individual behaviors so that the desired collective behavior emerges from their interactions [3].

1.1.3.2 Robustness, Scalability, and Flexibility

These traits are the main advantages of adopting a swarm approach in dynamic and uncertain environments that call for redundancy and adaptability [2], [14].

- **Robustness:** Redundancy and distribution allow operation to continue despite disturbances or failures of individuals, so that performance degrades gracefully: the loss of some members is compensated for by the others.
- **Scalability:** Enforcing local coordination enables effective operation from small groups up to thousands of agents without redesigning the software or hardware. The essential condition is that all interactions remain local.
- **Flexibility:** Self-organized structures support rapid re-tasking for diverse goals and environments with only minor software changes.

1.1.3.3 Limited Individual Capabilities (Sensing and Communication)

Individual robots are simple, small, and low-cost, which makes it possible to deploy large populations that achieve collective productivity through cooperation; a single robot is often incapable or inefficient at the main task on its own [2]. Local sensing and short-range communication restrict interactions to neighbors, which is vital for the scalability and stability of distributed coordination in swarms [1], [2].

1.2 Distinguishing Swarm Robotics from Other Paradigms

Swarm robotics differs from multi-robot systems, sensor networks, and multi-agent systems in terms of scale, control, and applications. Table 1.1 summarizes these differences [2].

Table 1.1. Comparison of swarm robotics with other paradigms [2].

Feature	Swarm robotics	Multi-robot system	Sensor network	Multi-agent system
Population size	Highly variable	Small	Fixed	Small
Control	Decentralized	Centralized / remote	Centralized / remote	Centralized / hierarchical
Homogeneity	Homogeneous	Heterogeneous	Homogeneous	Homogeneous / heterogeneous
Flexibility	High	Low	Low	Medium
Scalability	High	Low	Medium	Medium
Environment	Unknown	Known / unknown	Known	Known
Motion	Yes	Yes	No	Rare
Applications	Disaster relief, military, hazardous tasks	Transportation, sensing, robot soccer	Surveillance, protection	Resource management, control

1.3 Importance of Path Planning in Swarm Robotics

Path planning is a key function in the navigation of autonomous systems, and it becomes doubly important in swarms because of the large number of agents, decentralized control, and strict safety and efficiency requirements. In a swarm robotic system, the planner is the high-level decision layer that selects the sequence of waypoints each robot follows, and

its coordination logic must produce safe, reliable, and efficient collective motion under local sensing and communication and under real-time constraints [6], [14], [15]. A general classification of path planning methods, following [15], is shown in Figure 1.2.

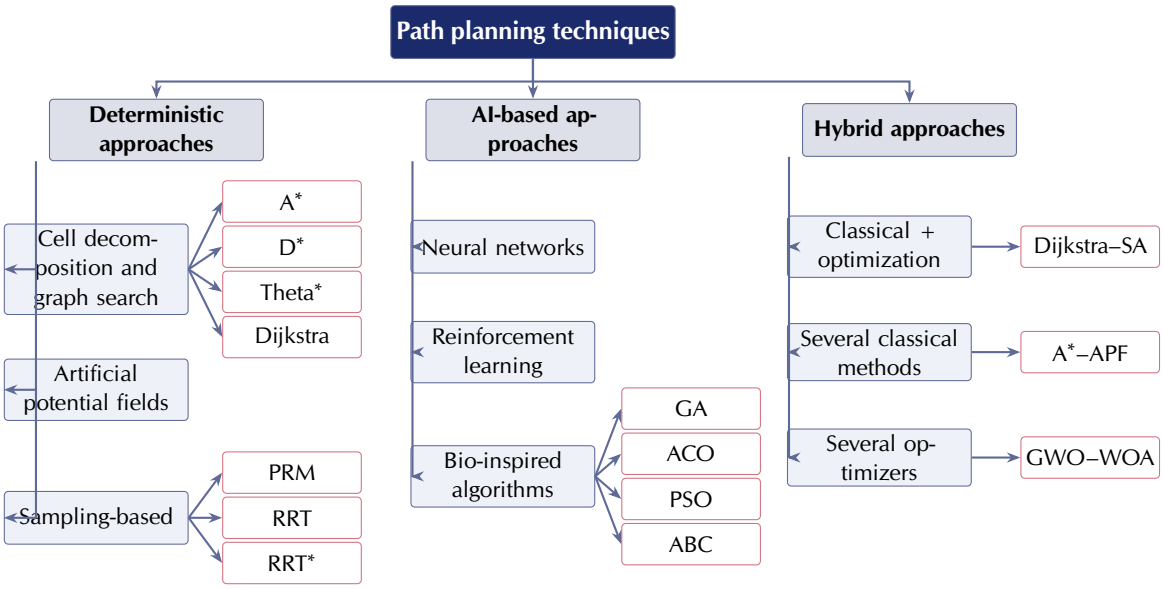
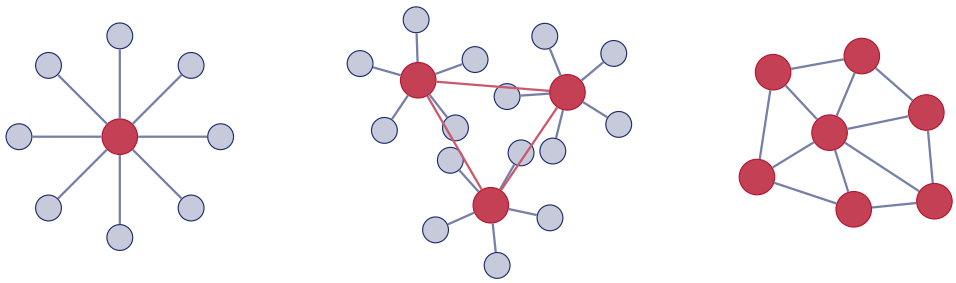


Figure 1.2. Classification of path planning methods (redrawn after [15]).

1.3.1 The Fundamental Challenge in Multi-Robot Systems

Multi-robot path planning requires integrating collision avoidance, optimality, dynamic feasibility, and team-level coordination, and it is generally treated as a combinatorial optimization problem, NP-hard in general, that scales poorly with team size and environmental complexity [14]. In the swarm setting, as the number of agents grows, the interaction graph and the coordination decisions expand sharply, and the computational and real-time challenges intensify well beyond those of single-robot planning [6], [14], [15]. In swarm robotics, each agent is responsible for planning its own path. In Figure 1.3, the processing units are shown in red; from left to right, the figure depicts centralized, decentralized, and distributed control, respectively. The goal of swarm robotics is to achieve the distributed architecture (the last case).



(a) centralized (b) decentralized (c) distributed

Figure 1.3. Control architectures in multi-agent robotics: (a) centralized, (b) decentralized, and (c) distributed. Processing units are shown in red.

1.3.1.1 Ensuring Safe, Reliable, and Smooth Motion

- **Safety and reliability:** Planning must guarantee safety with respect to static and moving obstacles in cluttered spaces while preserving group motion; this is critical in search-and-rescue and air-traffic scenarios, where failure is unacceptable. Relying solely on local formation or flocking breaks down in crowded areas and rough terrain, so explicit planning constructs such as safe corridors or tubes are needed to maintain cohesion and resolve conflicts [14], [15], [16], [17].
- **Smoothness:** Because of kinematic and nonholonomic constraints, the path must be a smooth curve, and the trajectory that follows it must respect bounds on velocity and acceleration in order to be trackable. This motivates optimization-based and hierarchical "plan-then-track" designs, in which a trajectory-planning stage turns the geometric path into a feasible motion profile. Virtual corridors and tubes and smooth trajectory parameterizations (e.g., B-splines and piecewise polynomials) are widely used for smooth, collision-free motion in real time [14], [15], [16].

1.3.1.2 Optimal Collision-Free Path Generation

- **Collision avoidance:** The planner must guarantee collision-free paths with respect to static obstacles, moving obstacles, and other robots, and in swarms it usually also maintains inter-agent separation, connectivity, and velocity consistency through constraints or cooperative control layers. A common mechanism is to enlarge the obstacles and shrink the robot to a point, which is formulated in the configuration space and simplifies the safety constraints [14], [15].
- **Optimal path selection:** Planning searches for an optimal or near-optimal path according to one or more criteria, such as path length, time, energy, smoothness, and robustness, which often leads to multi-objective formulations that trade off efficiency, safety margins, and dynamic feasibility [6], [14].

In swarms, optimality also encompasses collective measures such as group cohesion, throughput in narrow passages, and coordination costs under distributed execution.

1.3.2 Specific Challenges of Swarm Path Planning

Path planning for swarm robots combines the challenges specific to individual robots with a large scale, limited computational and energy resources, and the need for decentralized operation, which makes traditional centralized methods unsuitable for large teams. Consequently, algorithms must emphasize locality, distribution, and real-time responsiveness while preserving safety and efficiency guarantees [6], [14].

1.3.2.1 Computational Complexity with Large Numbers of Robots

Optimizing the trajectory of every robot in a swarm is expensive: the computational complexity of optimizing one parameterized trajectory ranges from $O\left(k(n_t, \varepsilon)\, n_t^2\right)$ to $O\left(k(n_t, \varepsilon)\, n_t^3\right)$, where n_t is the number of trajectory parameters and k is the number of iterations required to reach accuracy ε, and this cost is paid once per robot. This motivates specialized structures that curb such growth, such as the virtual tubes of Section 3.6 [16].

- **Combinatorial growth:** As the numbers of agents and obstacles increase, the decision space, the planning cost, and the risk of collision all grow rapidly [18]. In multi-agent reinforcement learning, the joint action space grows exponentially with the number of robots and their degrees of freedom, which is the familiar curse of dimensionality [19], [20].
- **Scalability of the architecture:** Centralized architectures suffer from heavy computational and communication loads and struggle with the dynamic behaviors and data traffic of large teams, whereas distributed methods spread the decision-making over the agents, reduce the burden on any single controller, and better preserve real-time responsiveness [14], [15], [21].

1.3.2.2 Energy Efficiency and Fault-Tolerance Requirements

- **Energy efficiency:** Swarm deployments in remote or harsh environments, where recharging is often impossible, require minimizing energy through shorter paths, smooth velocity profiles, efficient communication, and lightweight computation, so as to maximize endurance and mission success [6], [14], [15].
- **Fault tolerance and robustness:** The redundancy of the swarm must preserve performance under individual failures through task redistribution, and the swarm must remain functional under high failure rates in hazardous environments [6], [14].
- **Robust communication:** The planning and tracking layers must cope with communication failures, recover from network partitions, work with low bandwidth, and re-synchronize teams, so that collective objectives are maintained despite disturbances [15].

1.3.2.3 Handling Dynamic Targets and Environments

Real-world search-and-rescue and field operations take place in large, harsh, and highly dynamic scenarios, which require planners that adapt online to unknown maps, moving obstacles, and changing layouts, beyond the reach of static or purely local heuristics [13], [14], [15].

- **Mobile and dynamic targets:** Swarms must maintain coverage and tracking of targets that appear, move unpredictably, or increase in number, which requires continuous re-targeting and cooperative reassignment of motion goals [21], [22].
- **Real-time adaptation:** To avoid costly replanning, algorithms increasingly rely on incremental repairs of existing paths, hierarchical schemes, safe corridors or tubes, and learning in the style of centralized training with decentralized execution (CTDE), which update paths incrementally and generate smooth, collision-free actions in real time [16], [21], [23].

1.4 Objectives and Structure of the Report

1.4.1 Objectives

The objective of this report is to provide a structured and rigorous review of path planning in swarm robotics, with an emphasis on strategies that are based on swarm intelligence

and enabled by artificial intelligence (AI), which unlock scalable, decentralized systems for real-world applications. In particular, the report aims to:

1. **Establish the foundations:** Review the key concepts of swarm intelligence, namely self-organization, decentralization, emergent behavior, and local interaction, as they apply to robotic systems.
2. **Analyze control architectures:** Categorize the decentralized, hierarchical, behavior-based, and hybrid control methods used in swarm robotics and their effect on system performance.
3. **Explore path planning techniques:** Explain the principles, strengths, and weaknesses of deterministic, AI-based (including bio-inspired metaheuristics and learning), and hybrid planners when applied to decentralized coordination.
4. **Address computational challenges:** Examine how decentralized strategies keep planning tractable and scalable when the number of robots is large.
5. **Evaluate optimization metrics:** Detail the multi-objective metrics of swarm operation: short, safe, and smooth paths under energy and fault-tolerance constraints.
6. **Identify future directions:** Outline promising research avenues, including adaptive learning, human–swarm interaction, and operation in highly dynamic environments.

1.4.2 Structure

The rest of the report is organized as follows:

- **Chapter 2 – Fundamental concepts and methods in swarm control:** Swarm intelligence and self-organization, stigmergy, the control architectures (decentralized, hierarchical, behavior-based, and hybrid), communication and sensing, and the fundamentals of path planning, including objectives, metrics, and global versus local strategies.
- **Chapter 3 – Path planning algorithms for swarms:** A systematic review of deterministic (graph- and sampling-based and potential-field), AI-based (bio-inspired metaheuristics such as ACO and PSO, as well as learning), and hybrid methods, and of virtual-tube planning for large swarms.
- **Chapter 4 – Challenges and applications:** Dynamic and complex environments, scalability and robustness, task allocation and cooperative control, and the main application domains of swarm robotics.
- **Chapter 5 – Summary, conclusions, and future work:** The effectiveness of the approaches is summarized, the advantages of swarm robotics are stated, and directions for future research are outlined.

CHAPTER 2

Fundamental Concepts and Methodologies in Swarm Control

2.1 Swarm Intelligence and Self-Organization

Swarm intelligence (SI) is an artificial intelligence paradigm that explains how large groups of simple agents with limited capabilities, solely through local interactions and without centralized control, give rise to intelligent, robust, flexible, and scalable behavior at the collective level [8].

Self-organized natural phenomena in social insects—such as foraging by ants, nest construction by termites, and collective migration—have inspired SI methods that solve complex distributed problems through decentralized decision-making and indirect coordination, which makes these methods attractive for modern engineering systems [9].

2.1.1 Definition and Concepts of Swarm Intelligence

Swarm intelligence is an important branch of AI that studies the intelligence emerging from the interactions of a large number of simple agents, and it clearly illustrates the principle that "the whole is greater than the sum of its parts." Successful SI algorithms abstract the behaviors of social insects into computational procedures and control laws and apply them to optimization, decision-making, and distributed robotics. Emergent properties arise from nonlinear local interactions and feedback, producing macroscopic patterns such as trail networks, task allocation, and cooperative transport. These mechanisms translate into decentralized control of robot swarms and are used for tasks such as exploration, coverage, clustering/sorting, and construction/self-assembly [8], [24].

2.1.1.1 Decentralized and Self-Organized Problem Solving

Decentralization means solving complex distributed problems without a central controller or a global state, with sensing, actuation, and communication distributed among the agents to improve robustness and scalability [8].

Self-organization is the spontaneous formation of ordered global patterns from many local interactions based on simple rules; the environment often serves as an external memory and communication medium for these indirect interactions [9].

- **Stigmergy (indirect coordination):** Stigmergy is coordination through modifications of the environment: agents "write" cues (e.g., pheromone-like markers) that later "reader" agents use as the basis for their actions, which enables delayed, scalable, and decentralized communication.
- **Practical implementations:** These include virtual pheromones, deployed markers with simulated diffusion, robots that deliberately act as stationary beacons or repeaters for a while, and software agents that read from and write to a shared state [4].

2.1.1.2 Key Figures and Foundational Works

The conceptual foundation of swarm intelligence rests on a few works that bridged collective biological behavior and artificial systems:

1. **Beni and Wang:** introduced the term *swarm intelligence* in 1989, while discussing cellular robotic systems [25].
2. **Bonabeau, Dorigo, and Theraulaz:** analyzed social-insect behavior in *Swarm Intelligence: From Natural to Artificial Systems* (1999) and identified distributed functioning, flexibility, and robustness as the defining traits of swarm-intelligent systems [8].
3. **Kennedy and Eberhart:** proposed particle swarm optimization in 1995 and later consolidated the field in their book *Swarm Intelligence* (2001), which gave it much of its formal methodology [26], [27].

These analyses are still the theoretical basis on which the study of emergent intelligent behavior in multi-agent and robotic systems relies.

2.1.2 Principles of Self-Organization and Stigmergy

Self-organization explains the emergence of macroscopic order solely from local agent–agent and agent–environment interactions, without central control and without explicit global templates [8], [9]. It rests on four fundamental principles:

- **Positive feedback:** Recruitment and reinforcement amplify partially useful solutions and create a snowball effect (e.g., trail reinforcement in ants).
- **Negative feedback:** Resource depletion, evaporation, competition, or saturation stabilize the patterns and prevent runaway dynamics.
- **Amplification of fluctuations:** Individual randomness seeds exploration and enables the discovery of new solutions (e.g., new food sources).
- **Multiple interactions:** An abundance of local interactions creates the correlation and cohesion needed for stable macroscopic structures.

2.1.2.1 Indirect Coordination through Environmental Cues

Coordination can be direct (physical or visual contact) or indirect, through modifications of the environment; the latter is an instance of stigmergy. The environment acts as a shared, persistent external memory, allowing robots with local sensing to exchange information asynchronously and reach collective goals without global knowledge [8], [9].

Local decisions based on sensed cues drive global organization and allow agents with limited perception to build trails, allocate tasks, and construct structures. This delayed, cue-based communication improves scalability and robustness against failures or communication loss in large swarms [8], [9].

2.1.2.2 Stigmergy in Natural and Artificial Swarms

In natural systems, Grassé introduced stigmergy (from the Greek *stigma*, mark, and *ergon*, work) to explain nest reconstruction in termites: the building activity is regulated not by the workers themselves but by the configuration of the nest, since each building action turns the local configuration into a new stimulus that triggers the next action without any supervisor

[8], [28]. Nest cleaning and the clustering of corpses work the same way: the modified environment itself tells the other workers what remains to be done [8].

Ant foraging is another important example: pheromone deposition and evaporation, acting as positive and negative feedback, give rise to self-organized trail networks that discover and reinforce the shortest paths under geometric constraints [9].

There are two types of stigmergy:

- **Quantitative (continuous):** Responses vary with the intensity of the stimulus (e.g., pheromone concentration), which explains pillar construction and trail reinforcement.
- **Qualitative (discrete):** Distinct categories of stimuli trigger different actions, as in paper-wasp nest construction guided by local cell configurations [8].

Artificial implementations of stigmergy in swarms include:

- **Virtual pheromones:** Software or field representations simulate deposition, diffusion, and evaporation in order to form trails, provide coverage, and prioritize regions.
- **Embedded media:** Smart environmental beacons or markers encode stigmergic information and simulate diffusion to provide scalable guidance.
- **Physical agents as the medium:** Some robots become stationary relays or anchors and provide local memory and navigation cues in large swarms.
- **Software agents and shared states:** Agents write to and read from a shared layer or map and use it as an external memory for indirect coordination [4], [8].

Because the environment carries the information, stigmergy lowers the communication demands on each robot and helps the swarm stay scalable [1]. It has been used for path formation between a nest and a goal, where robots themselves form chains or act as beacons [29], for collective cleaning [30], and for self-assembly [2].

A canonical mathematical model of trail-based stigmergy is as follows. Ants choose between two branches with a pheromone-weighted probability $p(C_i)$, and the pheromone dynamics, with evaporation and deposition, lead to a bifurcation from symmetric to asymmetric traffic above a critical flux [9]. The path choice is given by

$$p\left(C_i \mid C_1, C_2\right) = \frac{(k + C_i)^\alpha}{(k + C_1)^\alpha + (k + C_2)^\alpha}, \qquad i = 1, 2, \tag{2.1}$$

where k and α depend on the species and the experimental setup. The pheromone evolves according to

$$\frac{dC_i}{dt} = \Phi\, p\left(C_i \mid C_1, C_2\right) - \mu\, C_i, \qquad i = 1, 2, \tag{2.2}$$

where Φ is the incoming flux and μ is the evaporation rate; this leads to multiple equilibria and to symmetry breaking beyond a critical value of Φ.

With the substitutions $C_i \to k c_i$, $t \to \tau/\mu$ and $\varphi = \Phi/(k\mu)$, and with $\alpha = 2$ (the value measured for the ant *Lasius niger*), the equilibria satisfy $c_1 + c_2 = \varphi$ and $(c_i - \varphi/2)(c_i^2 - \varphi c_i + 1) = 0$. Besides the symmetric "no-choice" state $c_1 = c_2 = \varphi/2$, two asymmetric states

$$c_{1,2} = \tfrac{1}{2}\left(\varphi \pm \sqrt{\varphi^2 - 4}\right) \tag{2.3}$$

exist when $\varphi > 2$, and the symmetric state then becomes unstable [9]. The collective response is measured by the asymmetry of the traffic over the two bridges,

$$\Delta C = \frac{|C_1 - C_2|}{C_1 + C_2}, \tag{2.4}$$

so that $\Delta C > 0$ indicates the emergence of a collective choice. Figure 2.1 plots these equilibria and the resulting asymmetry $\Delta C = \sqrt{\varphi^2 - 4}/\varphi$; the dots are obtained by integrating the dimensionless model numerically from a slightly perturbed symmetric state. Since the critical flux is proportional to the evaporation rate, the faster the pheromone evaporates, the more ants are needed before a choice emerges.

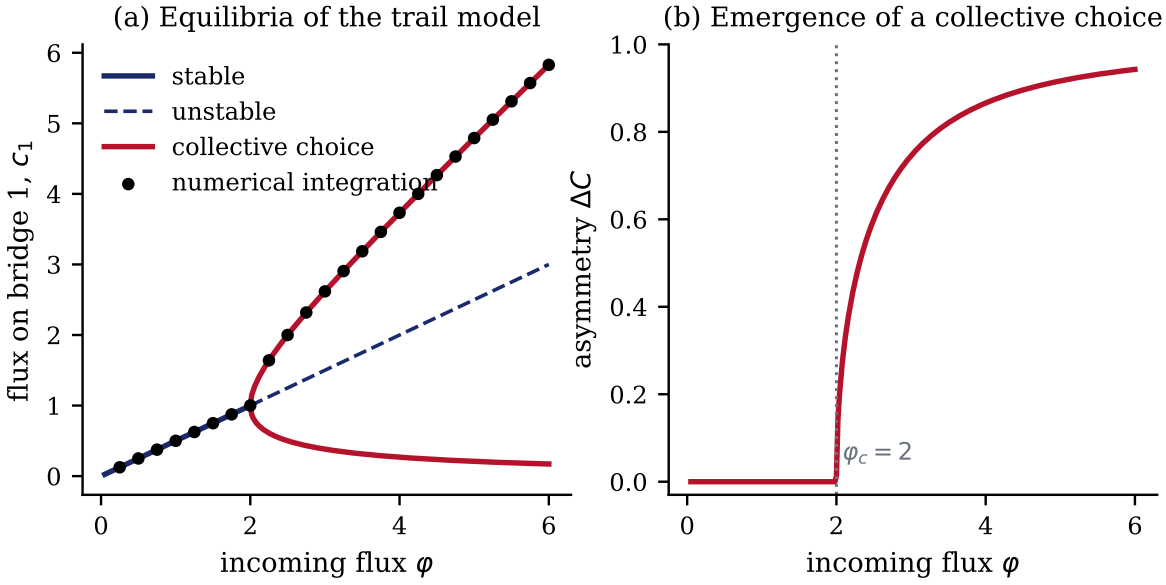


Figure 2.1. Pitchfork bifurcation of the trail model with $\alpha = 2$: (a) equilibria of the flux on bridge 1 as a function of the incoming flux φ, with numerical integration as dots; (b) the asymmetry ΔC, which becomes positive above $\varphi_c = 2$.

2.2 Control Architectures for Swarm Robotics

Control architectures in swarm robotics are designed to achieve robust, scalable, and flexible collective behavior; they emphasize decentralized decision-making, lightweight local communication, and formal mechanisms for convergence and safety in large groups of robots. The central challenge in swarm control design is that the control paradigm determines how information flows among the agents and how local interactions produce reliable macroscopic performance under uncertainty and failures [10].

Unlike classical robotics, swarm control favors distributed, self-organized schemes over monolithic global supervisors, in order to avoid bottlenecks, single points of failure, and scalability problems with hundreds or thousands of agents [1].

2.2.1 Decentralized Control

Decentralized control is the cornerstone of swarm robotics: each robot runs a simple, autonomous controller that relies on local sensing and neighborhood exchanges, and macro-

scopic performance emerges from concurrent local decisions without a central coordinator [6], [10].

2.2.1.1 Local Decision-Making and Adaptability

In decentralized architectures, each agent perceives only its own neighborhood, applies local rules, and updates in parallel with its peers; in this way, behaviors such as flocking, pattern formation, and exploration emerge without the need for a map or for broadcasting a global state [10]. Locality enables rapid adaptation: when the environment or the mission changes, the agents immediately adapt to the new local cues and drive a coherent macroscopic reconfiguration without global commands [1].

A typical local control law for agent i is based on the differences with its neighbors:

$$\dot{x}_i = \sum_{j \in \mathcal{N}_i} a_{ij} \left(x_j - x_i \right), \tag{2.5}$$

which can be written in vector form as

$$\dot{x} = -Lx, \tag{2.6}$$

where L is the graph Laplacian; convergence is guaranteed under standard connectivity assumptions [10].

Distributed flocking, by combining the simple rules of separation, alignment, and cohesion, or by imposing geometric constraints, maintains distances, headings, and shapes using only neighbor data and enables robust motion through unknown environments [11].

2.2.1.2 Distributed Algorithms and Scalability

Distributed swarm algorithms emphasize simplicity, locality, robustness, and parallelism, so that the complexity per robot stays low and performance is preserved as the team grows [6].

Centralized schemes often suffer from channel congestion and high latency, whereas distributed protocols naturally balance the load, tolerate faults, and exploit the concurrency of sensing and actuation [10].

- **Simplicity and finite state machines:** Controllers are often finite state machines (FSMs) or short reactive chains, so that their timing remains predictable [10].
- **Local-only communication:** Neighbor-to-neighbor exchange preserves scalability and avoids saturating a global channel, while still enabling consensus, formation, and task allocation over graphs [2].
- **Dynamic membership:** Agents may join or leave; the protocols are recomputed with the current neighbors and have no hard dependence on specific members [1].

2.2.1.3 Consensus Principles

Consensus is the agreement of robots on shared variables—such as reference frames, headings, goals, virtual leaders, or shape parameters—through local exchanges over a graph with adjacency matrix A and Laplacian L. Continuous-time average consensus follows $\dot{x} = -Lx$, and its discrete-time counterpart $x_{k+1} = (I - \varepsilon L)\, x_k$ with step size ε converges if the graph is connected and the spectral conditions hold [10].

Consensus achievement is also a collective decision-making behavior: the swarm must agree on one option among several, such as a path, a target, or a formation branch. It can be reached through direct communication, where robots exchange their preferences explicitly (for example by voting), or indirectly, where the decision emerges from cues such as the local density of robots [3], [10].

Practical controllers estimate distributed quantities using local filters, event-triggered updates, or time-varying weights, in order to reduce bandwidth and to track changing goals stably and quickly. Dynamic average consensus estimators, for instance, let every robot estimate collective quantities such as the abstract shape of the swarm without a leader [31].

An example of event-triggered multi-robot control based on the adjacency matrix $A = [a_{ij}]$ is

$$u_i(t) = \sum_j a_{ij} \Big[\alpha \big(x_j(t_j^s) - x_i(t_i^s) \big) + \beta \big(v_j(t_j^s) - v_i(t_i^s) \big) \Big], \tag{2.7}$$

which uses a triggering condition $g_i(t) > 0$ so that each robot updates and transmits only when necessary, and which guarantees position–velocity consensus through Lyapunov analysis [32].

2.2.2 Hierarchical Control Architectures

Hierarchical architectures sit between purely centralized and purely distributed control: one or more local coordinators organize the robots into functional clusters, which makes them a subset of decentralized architectures [33]. They are used to manage complexity, to integrate diverse functions in heterogeneous systems, and to give guarantees on global quantities in tasks such as formation flight or self-reconfiguration. The design usually proceeds top-down, from global objectives to components small enough for the individual agents [3].

2.2.2.1 Multi-Level Control for Formation Generation

In multi-level formation control, a high-level guidance layer handles global objectives, coordination, and the outer loops, often through a virtual leader that provides the reference trajectory of the formation; a low-level layer runs the inner loops (attitude control and local trajectory tracking) under tight real-time requirements. The followers adjust their positions and velocities using the virtual leader and their neighbors [34], [35]. This split compensates for the limited processing power of embedded platforms and makes it easier to implement high-level algorithms. In the same spirit, algorithms are often developed on a hierarchy of models of increasing realism before they are ported to real robots [10].

2.2.2.2 Example: The Five-Layer Software of SambotII

The self-assembly modular robot SambotII upgraded the computing and perception of its predecessor with an Intel x86 dual-core processor for computation-heavy tasks such as image processing and path planning. Its software is organized in five layers with uniform abstract interfaces between them: a hardware abstraction layer that hides the motor, encoder, and camera details, a module abstraction layer, an operation layer, a behavior layer, and a task layer. The layering improves programming efficiency, reliability, and reuse [36].

2.2.2.3 Hierarchical Action Selection Process (HASP)

HASP is a behavioral mechanism that coordinates a set of elementary controllers in a hierarchical and flexible way, both inside each robot and across the group. A higher-level behavior hands control to a lower-level one only when it issues a *refuge command*, which moves the robot toward a reliable *refuge position*; this keeps the resulting actions viable even in highly dynamic multi-robot interactions. Adding a mechanism for fusing actions gives the hybrid and hierarchical variant (Hybrid-HASP), in which several behaviors can act at the same time; both were evaluated on the cooperative box-pushing task [37].

2.2.2.4 Multilevel Topology Design for Shape Control

For shape control, robots can be organized in concentric levels that grow outward as the swarm grows, a structure inspired by the hexagonal close-packing of atoms, which aggregates robots in the least space and scales naturally. The robots first converge to the desired region and then form the shape level by level; different shapes are produced by allocating robots to allowed positions and marking the others as virtually occupied. A controller built from several potential forces drives the robots to their positions without collisions, and a dedicated shape-regulation force removes the local minima in which robots would otherwise get stuck [37].

2.2.3 Behavior-Based Control Methods

Behavior-based control (BBC) realizes swarm intelligence through bottom-up reactive behaviors rather than global plans, keeping the system simple, robust, and responsive in unknown or dynamic environments [10].

BBC is aligned with local sensing/actuation and emergent coordination, which are hallmarks of scalable swarm operation, and it is the most common way of designing swarm robotic systems: the designer implements, studies, and refines the behavior of each robot, bottom-up, until the desired collective behavior emerges [3]. Brooks' subsumption architecture is the classical example of layering behaviors by priority (e.g., obstacle avoidance above exploration): the behaviors run concurrently, but higher-priority behaviors suppress lower-priority ones and release them once the stimulus disappears [38]. Iterative refinement at the robot level—tuning behaviors and thresholds—produces reliable collective patterns without a global model or explicit global synchronization [6].

2.2.3.1 Reactive Behaviors for Emergent Intelligence

Reactive controllers map the current sensor inputs and a minimal internal state to actions through condition–action rules and avoid complex world models, so that they remain replicable across large teams [10]. Emergent behaviors such as flocking, dispersion, trail formation, and collective decision-making arise from many local interactions rather than from a single global planner [24].

Artificial potential fields (APFs) superimpose attractive forces toward goals and repulsive forces from obstacles and peers to provide safe distributed navigation without global supervision, although parameter tuning and local minima require care.

2.2.3.2 Finite State Machines in Controllers

FSMs are widely used in swarm controllers: states encapsulate behaviors, and transitions are triggered by sensor events, timers, or messages, which ensures predictable responsiveness with a simple architecture [10]. Probabilistic FSMs (PFSMs) add stochastic transitions to model noise and variability and facilitate the analysis of convergence and robustness in aggregation, chaining, or allocation tasks [10]. Two kinds are used: in one, the transition probabilities stay fixed; in the other, they change with inputs from the environment and from other robots [3]. Composing simple automata into a model of the whole system is also the basis of supervisory control theory, which has been applied to swarm robotics to synthesize controllers with formal guarantees [39].

2.2.3.3 Response Threshold Models

Response threshold models link an agent's probability of switching tasks to the intensity of a stimulus, producing decentralized and flexible task allocation in which task sequencing emerges from the accumulation of collective stimuli [3], [8]. The standard nonlinearity is

$$p(s) = \frac{s^\beta}{s^\beta + \theta^\beta}, \tag{2.8}$$

where s is the stimulus, θ is the threshold, and β controls the sensitivity. As s exceeds θ, the probability of acting rises sharply, and coordination emerges without an explicit leader.

2.2.4 Hybrid Control Methods

Hybrid controllers combine behavior-based control with systems-theoretic guarantees to handle missions that require both fast local responses and guaranteed macroscopic performance [10]. A common pattern embeds reactive rules within a statistical or model-based regulator that governs swarm-level quantities (e.g., the mean position or the density variance), thereby preserving autonomy while constraining the macroscopic behavior [7].

This synthesis retains robustness and scalability while achieving explicit objectives such as tracking, containment, and safety margins through provable controllers layered around the reactive behaviors.

2.2.4.1 Combining Behavior-Based and Systems-Theoretic Approaches

The goal is to retain the qualitative strengths of BBC—simplicity, robustness, and fast reaction—while adding formal control to regulate collective metrics and stability [10].

Statistical or macroscopic controllers can constrain the swarm's center of mass and dispersion, while reactive layers ensure collision avoidance, obstacle negotiation, and local safety, thus reconciling global guarantees with local autonomy [7].

Formal tools include Lyapunov functions, graph-theoretic convergence results, and density-based partial differential equation (PDE) formulations for quantifying and bounding macroscopic performance under distributed execution [10].

2.2.4.2 Applications: Underwater Mine Countermeasures

Hybrid control excels underwater, where uncertain currents, limited communication, and safety-critical sensing require both local reaction and macroscopic coordination [40]. For

underwater mine countermeasures, a subsumption architecture was embedded in a statistical controller that regulates the mean and the variance of the swarm's position. The subsumption layer orders the robot's tasks so that mine and obstacle avoidance come before aggregation, separation, and random search. When one robot has to swerve around an obstacle, the statistical layer makes the rest of the swarm regroup, so the individual gets the latitude it needs while the swarm mean stays where it should; the combination performed better than either method alone [40].

Event-triggered and consensus-based schemes reduce bandwidth while maintaining formation and tracking [32], and reactive potential fields provide on-the-spot avoidance of obstacles and hazards.

In such implementations, the macro layer regulates group objectives (e.g., the mean trajectory of the formation), while the micro layers execute avoidance, separation, and target pursuit, so that the autonomy and survivability of each autonomous underwater vehicle (AUV) are preserved.

A worked example of this layering, cooperative target hunting by a team of AUVs with an improved potential field, is described in Section 3.3 [41].

2.2.4.3 Hybrid Systems with Real and Virtual Agents

Validating controllers for large swarms experimentally is slow and expensive, so many macroscopic methods (for example continuum models that approximate the swarm by partial differential equations) are tested only in simulation. A mixed reality platform offers an intermediate step: a few real differential-drive robots share the arena with many virtual agents, a multi-agent version of hardware-in-the-loop testing. In one such platform, four robots and 96 virtual agents formed a swarm of 100 that interacted through a repulsive kernel; a continuification-based controller computed the desired positions and velocities, which the robots tracked with their embedded controllers, and the swarm achieved the desired density in the plane [42].

2.2.5 Virtual Structure and Virtual Linkage Approaches

Formation control can be framed as leader–follower, behavior-based, or virtual structure (VS) control. The virtual linkage (VL) approach generalizes VS to allow precise control with reconfiguration through virtual joints and links. VL treats the formation as a mechanical structure of virtual links connected by joints whose angles determine the shape of the pattern, and it enables reshaping by controlling the trajectories of these angles [43].

2.2.5.1 Virtual Linkage and Degrees of Freedom

In VL, the desired shape is parameterized by the virtual joint angles Θ_d, and the number of joints equals the number of degrees of freedom available for reconfiguring the pattern; adding an expansion/contraction vector λ_d scales the link lengths for passing through narrow passages. Because the robots follow only local virtual-linkage references, reshaping is achieved by broadcasting a small set of parameters to a few link-level controllers, which reduces communication compared with recomputing the slot of every member [43].

VL decouples precise control from global design, offers a compact command space for frequent changes of shape, and keeps execution distributed at the robot level. Figure 2.2

illustrates how a virtual linkage works. In a serial linkage, the number of degrees of freedom equals the number of virtual joints, that is, the number of virtual links minus one, so more links allow more formation shapes [43].

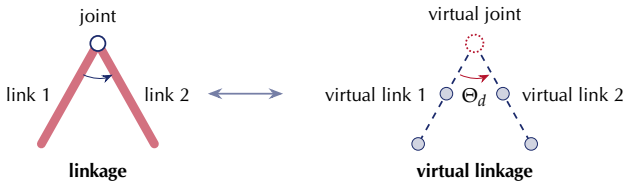


Figure 2.2. A mechanical linkage and its virtual counterpart, the virtual linkage (redrawn after [43]).

2.2.5.2 The Virtual Structure as a Special Case

A virtual linkage whose joint angles are held constant behaves as a single rigid body, and a linkage with a single link ($k = 1$) is exactly a virtual structure, with no ability to reconfigure. The two differ sharply in communication: to change shape, a virtual structure must redesign and transmit new relative positions to every agent, whereas a virtual linkage broadcasts only the joint angles to the k link controllers. The analysis in [43] shows that the linkage needs less than a third (33.3%) of the bandwidth of a virtual structure to reconfigure a formation, which makes it attractive when the shape must change often.

2.2.5.3 Decentralization and Open Directions

The first virtual-linkage implementation was hierarchical: a linkage server sent states to link servers, which forwarded them to the members of each link. This is better than a centralized virtual structure but still limits scalability and is vulnerable to bandwidth limits and lost messages. A decentralized version removes the servers: every agent runs the same consensus algorithm, computes its own instance of the linkage states from information exchanged with its neighbors, and decides its motion on its own, so no roles have to be assigned [44]. The authors point to formation feedback, the dynamic generation of formation patterns, and route planning in unknown environments as the next steps [43], [44].

2.3 Communication and Sensing in Swarms

Communication and sensing are the main channels through which collective intelligence manifests itself in swarms; local exchange and interaction mirror the decentralized organization found in nature. The chosen control architecture fixes the topology of information exchange and hence the performance, so reliable, fault-tolerant communication is essential for effective coordination at scale [2], [24].

2.3.1 Local Interactions and Limited Communication

Swarm controllers place decision-making at the agent level and, with simple computation and minimal sensors on each robot, achieve scalability, robustness, and flexibility through parallel, neighbor-based interactions. This design removes any dependence on a global state and keeps working in the presence of motion, topology changes, and partial failures [1].

2.3.1.1 Local Sensing Capabilities

Swarm robots are usually homogeneous and resource-constrained, so sensing is deliberately local and minimal to keep cost and energy consumption low while still enabling distributed behaviors. Consequently, each robot acts on instantaneous observations and simple rules, and the range and bearing to nearby neighbors are vital [1].

- **The need for local sensing:** Agents perceive nearby robots and obstacles and act autonomously using only these measurements and concise rules, which often involve estimating the distances to neighbors for the interaction rules.
- **Sensor types:** Common modalities include proximity and range sensing (infrared (IR), ultrasonic), cameras and light sensors, inertial measurement units (IMUs), microphones, and simple environmental sensors. Very small platforms use IR LEDs and photodiodes for local communication and relative localization, and the Kilobot uses downward-facing IR reflected off the floor for limited-range communication and localization.
- **Purpose:** Local sensing makes it possible to follow local rules and interact with neighbors, which is essential for decentralized coordination and the emergence of group behavior [10].

2.3.1.2 Challenges in Predicting Simultaneous Interactions

Decentralized swarms face predictability challenges because abundant local interactions create nonlinear, emergent dynamics that are difficult to control and predict in advance. This complexity stems from micro-to-macro coupling, in which macroscopic behavior emerges from microscopic decisions made under limited information and a changing topology [3], [45].

- **Unpredictable emergence:** Undesired global patterns can emerge from large numbers of simultaneous local interactions, complicating formal guarantees and certification.
- **Cognitive load:** The cognitive difficulty for a human operator does not remain a constant $O(1)$; dependencies and incomplete information under bandwidth and latency constraints increase the supervision burden.
- **Topology dependence:** Motion reshapes the graph and risks fragmenting it into disconnected components, unless the algorithms preserve connectivity and robustness under dynamic neighbor sets.
- **Information propagation:** Building global maps or shared situational awareness from local observations is constrained by short-range, limited-bandwidth links and requires efficient multi-hop dissemination in highly dynamic networks.

2.3.1.3 Short-Range Communication

Swarm design deliberately restricts communication to short ranges to preserve scalability and robustness by avoiding channel saturation and single points of failure. Local links enable massive parallelism and redundancy and keep the hardware of each robot simple and inexpensive [1].

- **Scalability constraint:** Global channels scale poorly as the team grows, so protocols enforce neighbor-only exchange to keep communication costs bounded and decentralized.

- **Communication modalities:** Both explicit links (peer-to-peer radio) and implicit ones (stigmergic or environmental cues, physical contact) support distributed coordination under local rules.
- **Radio frequency (RF):** IEEE 802.15.4-class radios on platforms such as the e-puck are often configured with power control for very short ranges (down to about fifteen centimeters) to enforce locality [30].
- **Infrared (IR):** Short-range messaging and proximity sensing, including mutual ranging and neighbor discovery, as in the s-bot/Swarm-bot systems [4], [46].
- **Sound/ultrasonic:** Acoustic links aid localization and messaging and can be combined with RF triggering for GPS-free distributed operation on land and underwater; the e-puck, for example, carries microphones and a speaker that can be used for acoustic messaging [30].

2.3.2 Communication Protocols and Network Topology

Swarm performance depends strongly on the communication architecture and on the network topology it induces; together, they determine how information flows, how quickly consensus forms, and how resilient coordination is to faults. The co-design of protocol and topology must account for motion-induced changes and bandwidth constraints in order to maintain stability and throughput [2].

2.3.2.1 Decentralized Communication Protocols

Decentralized protocols allow robots to exchange data without a central broker, producing robust, scalable, low-latency coordination. The categories include direct messaging, indirect stigmergy, and direct physical interaction, which are often combined for redundancy and adaptability [1], [3].

- **Direct peer-to-peer communication:** Explicit exchange of states, events, or votes reduces latency and enables rapid reactions to local contingencies.
- **Indirect communication (stigmergy):** Agents leave environmental markers that shape subsequent decisions, providing an inexpensive, delay-tolerant flow of information.
- **Direct interaction:** Physical contact or force sensing conveys local state cues and constraints and is useful in tight formations and cooperative manipulation.
- **Design goals:** Local decision-making with adaptive protocols, multipath redundancy, and distributed communication functions reduces single points of failure and preserves functionality under partial outages.

2.3.2.2 Graph Theory and Network Structure

Graph theory models robots as nodes and communication links as edges, enabling analysis based on the adjacency matrix A and the Laplacian $L = D - A$ (where D is the degree matrix) to support convergence proofs for distributed controllers. The choice of graph and its connectivity determine the stability and convergence rates of consensus, formation, and distributed estimation [47], [48].

- **Graph types:** Undirected graphs model bidirectional exchange and directed graphs (digraphs) model one-way flow; the spectral properties of L shape the convergence and

robustness margins.

- **Proximity graphs:** The limited sensing and communication range induces proximity topologies such as the r-disk graph $G_{\text{disk}}(r)$, in which an edge exists between two robots that are closer than r.
- **Leader–follower structure:** A leader guides the collective motion and the followers track it using the leader's state and data from their neighbors; the leader usually receives nothing from the followers, and convergence in directed networks requires a directed spanning tree rooted at the leader [49].

2.3.2.3 Network-Theoretic Approach to Information Flow

The network-theoretic perspective views the swarm signaling network (SSN), the channel through which behavioral information flows, as a temporal, adaptive network whose topology co-evolves with the motion of the agents, coupling communication and control. Like any real channel, it has a finite capacity C set by noise and bandwidth B, which bounds the achievable performance and calls for an information-theoretic view of swarm design [50].

- **Structural properties:** Many biological and engineered swarms exhibit small-world traits—short paths and high clustering—which support the fast and robust propagation of behavioral signals, such as the very fast evasive maneuvers of natural swarms.
- **Network controllability:** Analysis can identify the minimal set of driver agents needed to steer the swarm dynamics and can inform the placement of leaders, beacons, or gateway nodes.

2.3.3 Sensor Models and Data Acquisition

Sensing, alongside actuation and computation, is a core subsystem; context awareness is obtained by fusing environmental and inter-agent measurements, including communication signals interpreted as sensors of the network state. The key challenge is to fuse heterogeneous, local sensors into robust cooperative behaviors and shared maps under tight resource constraints [1], [10].

2.3.3.1 Sensor-Based Scene Understanding

Scene understanding combines representation and data association to produce actionable context from on-board measurements. Three complementary levels of representation are common in swarms with heterogeneous payloads and different roles [13]:

- **Appearance level:** Local and global visual features support state estimation and visual place recognition for lightweight localization.
- **Geometry level:** LiDAR and RGB-D sensors provide 3-D points and depth; RGB-D is sensitive to lighting, whereas LiDAR is costly and heavy, which justifies deploying them selectively in mixed swarms.
- **Semantic level:** Object detection, multi-target tracking, and semantic segmentation improve navigation and make simultaneous localization and mapping (SLAM) robust to dynamic changes.
- **Output artifacts:** Semantic local maps and scene graphs provide high-level constraints for distributed localization and cooperative planning.

2.3.3.2 UAV Sensor Models for Target Tracking

- **Direction-finding sensors:** The most common choice for distributed target tracking with UAV swarms; they typically fuse a photoelectric or infrared imager with an ultrasonic or laser rangefinder to measure both the range and the azimuth of the target [51].
- **Emitter localization:** For radio sources, UAVs carry time-difference-of-arrival (TDOA) receivers, and the configuration and trajectories of the swarm are chosen with the Cramér–Rao lower bound (CRLB) to improve the accuracy and stability of the estimate [52].
- **Fusion for integrity:** For relative navigation of several UAVs, double-differenced GNSS pseudoranges have been fused with ultra-wideband (UWB) ranging to detect and exclude faulty observations and improve integrity monitoring [53].
- **Chemical plume tracing:** Quadcopters for plume tracing carry chemical sensors to measure concentration and ultrasonic sensors to hold their altitude, and the effect of the propellers on the chemical puffs has to be accounted for [37].

2.3.3.3 Acoustic Sensing for Mutual Identification

- **Mutual acoustic identification:** e-puck robots can detect the presence of their neighbors with a single microphone, which lets them identify one another without vision or dedicated ranging hardware [54].
- **Acoustic trilateration:** A localization system based on acoustic trilateration measures the time between an RF pulse and the matching acoustic pulse on dedicated sensor boards, using transducers pointed down at parabolic cones; it locates neighbors in 2-D without global information and can be extended to 3-D [30].
- **Ultrasonic sensors:** Standard on many swarm robots for non-contact obstacle detection, although their range is short (a few meters within a narrow cone) [37].

2.4 Basic Concepts of Path Planning

Path planning is a fundamental problem in robotics and autonomous systems because it provides the high-level decision-making layer that selects a continuous sequence of spatial waypoints for the robot to follow, and it directly affects autonomy, safety, and mission efficiency. In swarm settings, path planning is intertwined with the decentralized control and communication layers, and it determines strategic routes that respect local sensing limitations while optimizing global mission criteria such as time, energy, and robustness [6].

2.4.1 Definition of Path Planning

Path planning is the problem of computing a feasible—and preferably optimal—path that moves a robot from an initial state to a goal state in an environment while respecting geometric, kinematic/dynamic, and safety constraints. As the field has matured, the goal has shifted from finding "any path" to finding multi-criteria optimal paths that reduce travel time, energy consumption, and risk, while improving smoothness and robustness for real-world deployments [6].

2.4.1.1 Generating Collision-Free Paths

A fundamental requirement is collision avoidance: the path must avoid static and moving obstacles and maintain safety margins along its entire length. In the configuration-space formulation, complex robot shapes are handled by "inflating" the obstacles or "shrinking" the robot to a point, so that collision tests reduce to point-in-free-space checks and an efficient search over admissible nodes and edges becomes possible [10].

Once path planning has delivered a collision-free geometric path, motion planning assigns time-parameterized velocity and acceleration profiles $v(t)$ and $a(t)$ that are consistent with the dynamic constraints, so that the path is tracked smoothly and safely. Geometric safety can be enforced by inflating the obstacles by the robot's footprint, which turns robot–obstacle interference into the separation of a point from the inflated obstacles in the configuration space [6], [10].

A typical tracking pipeline optimizes a timing law $s(t)$ along the curve $\gamma(s)$ so that the bounds $\|\dot{\gamma}\| \leq v_{\max}$ and $\|\ddot{\gamma}\| \leq a_{\max}$ are respected, thereby separating spatial feasibility from dynamic feasibility for embedded controllers [6].

2.4.1.2 A Constrained Optimization Problem

Path planning is a constrained optimization search in which the objective may combine distance, time, smoothness, energy, and robustness, while feasibility must respect kinematic, dynamic, and safety constraints; many formulations are NP-hard in the general case. Multi-objective criteria are common in multi-robot systems and require Pareto reasoning or scalarization to balance conflicting objectives, such as the shortest time versus the lowest energy or the largest safety margin [6], [10].

The usual constraints are the robot's geometry, kinematics, and dynamics, the obstacles (collision-free motion is a hard constraint), and, in multi-robot missions, time windows for the tasks [14]. A representative cost function is

$$J(\gamma) = \alpha_1 L(\gamma) + \alpha_2 T(\gamma) + \alpha_3 S(\gamma) + \alpha_4 E(\gamma), \tag{2.9}$$

where L is the length, T is the travel time, S is a surrogate for smoothness such as curvature or jerk, and E is the energy; the coefficients α_i are tuned to the mission requirements. Swarm intelligence methods such as PSO and ACO are widely used to approximate global optima under complex constraints in high-dimensional configuration spaces or dynamic environments, often smoothing the candidate paths with B-splines to respect nonholonomic constraints [55].

2.4.2 Global versus Local Planning

Planners are classified by the scope of their information: global planners assume a known map and compute the path from start to end, whereas local planners act reactively and rely on real-time sensing to cope with unknown or dynamic obstacles. Practical systems are generally hybrid: global paths serve as a prior, while local planners ensure safety and reactivity when the map is incomplete or changes rapidly [6], [10].

2.4.2.1 Global Planning

Given a prior map, global planning computes the complete optimal path from the origin to the destination, either by discretizing the free space and solving a graph search or by sampling-based planning that efficiently captures the connectivity of the space. Classical graph methods such as Dijkstra's algorithm and A* guarantee optimality on weighted grids or roadmaps under admissible heuristics, but they incur replanning overhead when the environment changes [10].

A* uses a heuristic $h(n)$ with $f(n) = g(n) + h(n)$, and optimality is preserved when h is admissible and consistent; this is effective for static maps, but updates are needed when obstacles move. Sampling-based planners such as the probabilistic roadmap (PRM) and the rapidly-exploring random tree (RRT) family capture the connectivity of the free space without exhaustive gridding and scale to high-dimensional configuration spaces, although in their basic forms they rely on asymptotic rather than exact optimality [6].

2.4.2.2 Local Planning

Local planners rely on on-board sensing to generate admissible motions in real time and, with minimal model assumptions, focus on obstacle avoidance and progress toward the goal. Reactive approaches such as artificial potential fields superimpose an attractive potential toward the goal and repulsive potentials around the obstacles, providing low-computation navigation; without enhancements, however, they suffer from local minima and oscillation and cannot guarantee convergence to the goal [18], [24]. When so little is known in advance that no path can be precomputed, the problem is sometimes called path *finding* rather than path planning.

The APF law takes the form

$$U(x) = U_{\text{att}}(x) + U_{\text{rep}}(x), \tag{2.10}$$

with the control $u(x) = -\nabla U(x)$. Enhancements include navigation functions, randomization, and local escape maneuvers to mitigate local minima. In dynamic scenes, receding-horizon variants and lightweight velocity-obstacle/optimal reciprocal collision avoidance (ORCA) constraints can be layered on top of the reactive fields to guarantee collision-free motion while preserving responsiveness [6].

2.4.3 Objectives and Metrics

Swarm path planning is usually multi-objective: it trades off length, smoothness, time, energy, and robustness while guaranteeing safety and respecting platform limitations across many agents. Designing sound metrics is critical, because swarm missions often lack a single universal optimum and require problem-specific combinations of objectives and constraints.

2.4.3.1 Optimizing Length, Smoothness, and Efficiency

Minimizing the path length and travel time reduces energy use and improves throughput, especially when mission duration and battery budgets are tight. Smoothness is essential for nonholonomic platforms: B-spline or polynomial smoothing reduces curvature and jerk, so that actuator limits are respected and tracking errors in motion generation are reduced [55].

For multi-robot task allocation, efficiency is often measured by the makespan, the time the whole team needs to finish all tasks [14].

2.4.3.2 Maximizing Information Value in UAV Missions

In unmanned aerial vehicle (UAV) tracking and surveillance, the objectives emphasize information value, revisit rate, and detection probability while respecting sensing and communication limits. In passive emitter localization, the waypoints are optimized to minimize the trace of the Cramér–Rao lower bound (CRLB), yielding favorable time-difference-of-arrival (TDOA) geometries for accurate estimates under range and angle constraints [52]. For persistent surveillance, a convenient objective is the maximum "age" of the cells, the time since each cell was last observed [56].

Information-driven planning balances observation quality against network constraints; decentralized model predictive control (MPC) formulations often maximize the Fisher information or the joint entropy, subject to inter-UAV spacing constraints that keep communication reliable. CRLB-based planning constructs sets of waypoints that preserve the observability of the estimator and a bounded covariance, translating geometry directly into localization-accuracy objectives [51], [52].

2.4.3.3 Reducing Computation Time and Energy Consumption

Real-time swarm deployment demands low computational latency and energy-aware routing in order to extend mission duration with limited on-board resources. Energy-oriented planning minimizes the propulsion and sensing/communication loads, while decentralized receding-horizon methods such as PSO reduce the solution time and scale to large numbers of agents [6], [55].

In a search over a 50×50 km area, a decentralized receding-horizon PSO planner did not quite match the centralized one but needed only half of its computation time, which favors on-board execution and rapid team replanning [56]. In dynamic environments, RRT-based replanners graft new waypoints near conflict regions instead of regenerating the whole path, preserving the rest of it and reducing computation time [23]. A multi-objective ACO planner for monitoring photovoltaic parks adjusts pheromone evaporation according to energy use; compared with a generic ACO, it reduced the travel time by 22% on average and increased the inspection coverage by 28% [57]. Not every method scales this well: mixed-integer quadratic programs coordinate three-dimensional motion and collision avoidance for micro quadrotors but with fast-growing run times [58], and distributed MPC carries a heavy computational load [51].

CHAPTER 3

Path Planning Algorithms for Swarm Robots

This chapter reviews bio-inspired optimization methods that have been adapted for decentralized, multi-robot path planning, emphasizing their natural inspirations, their operational rules, and their adaptations to robot collectives for robust and scalable navigation in dynamic environments.

Swarm intelligence approaches are popular in multi-robot settings because simple local rules and indirect communication lead to emergent macroscopic behaviors that remain robust to failures and scale to large populations without centralized control [6].

3.1 Bio-Inspired Algorithms

By mimicking foraging, flocking, and colony organization, bio-inspired algorithms navigate complex spaces without a global model of the environment, thereby achieving fast convergence and the ability to escape local minima for NP-hard combinatorial objectives in multi-robot teams [14].

These methods rely on decentralized decisions by the agents, local perception, and direct interaction or stigmergic signals, which together improve robustness, scalability, and real-time responsiveness under the uncertainties common in multi-robot deployments [6]. Unlike planners that depend on an explicit model of the environment, they let each robot act like an individual in a natural collective, which favors decentralized decisions [13].

3.1.1 Ant Colony Optimization (ACO)

ACO is a classical swarm intelligence method in which artificial ants deposit and follow pheromone while constructing paths, steering the search toward promising routes in large combinatorial spaces and in the motion graphs of robots [55].

Its core idea is positive feedback through pheromone reinforcement, combined with evaporation to balance exploitation and exploration, which makes ACO well suited to high-dimensional routing and discrete motion planning problems in robotics.

3.1.1.1 Inspiration from Ant Foraging Behavior

- **Pheromone mechanism:** Real ants stochastically prefer paths with higher pheromone concentrations, so shorter paths accumulate pheromone faster and are chosen more often; the algorithm reproduces these dynamics of pheromone deposition and evaporation [55].
- **Stigmergic foundation:** Trail laying and trail following constitute stigmergy—indirect coordination through modification of the environment—which in robotics can be implemented with virtual markers or distributed maps for decentralized navigation [59].

Figure 3.1 illustrates how ants select paths according to the pheromone concentration. At first, there is no pheromone on either branch, so the ants choose at random and the share on the short branch fluctuates around one half. Because a round trip over the short branch is

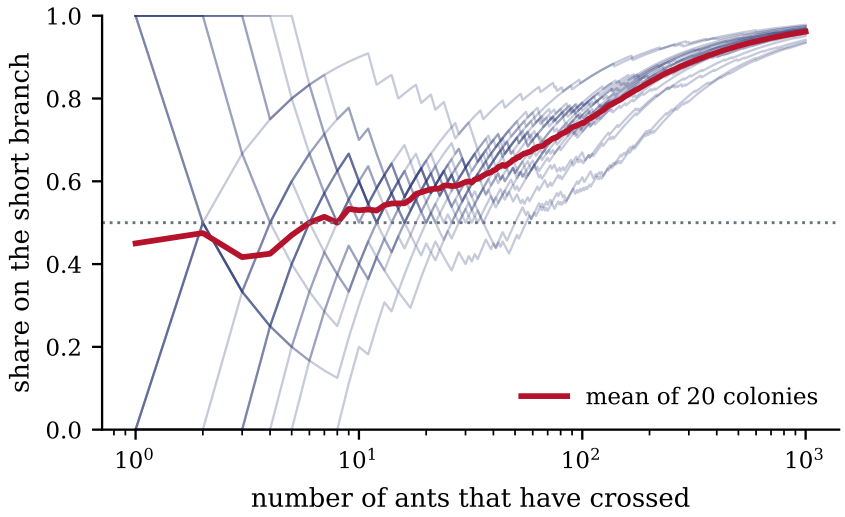


Figure 3.1. A simulated double-bridge experiment: share of ants on the short branch for 20 colonies (thin lines) and their mean (thick line). Choices follow Equation (2.1) with $k = 20$, $\alpha = 2$, and the long branch is twice as long, so it receives half the pheromone per crossing.

quicker, it is reinforced more strongly per unit time (modelled here by a larger deposit), and the pheromone difference grows. This positive feedback makes later ants prefer the short branch, and after about a thousand crossings every simulated colony sends more than 90% of its traffic over it. These dynamics, which reproduce the classical double-bridge experiments with real ants [8], form the basis of ant colony optimization.

3.1.1.2 Variants and Applications (AS-TSP, ACS-TSP)

- **Ant System (AS-TSP):** The first ACO algorithm, proposed for the traveling salesman problem (TSP), which introduced the pheromone-guided constructive building of solutions but was later surpassed by improved variants [60].
- **Ant Colony System (ACS-TSP):** ACS strengthens AS with local pheromone updates, usually integrates local search such as 2-opt and 3-opt, and achieves leading performance, especially on the asymmetric TSP [8], [61].
- **Other applications:** ACO variants have addressed quadratic assignment, sequential ordering, vehicle routing, and graph coloring problems. In robotics, ACO has been used to optimize the parameters of the membership functions of fuzzy controllers and, combined with fuzzy cost functions, for the navigation and path planning of mobile robots [55]. Together with PSO, ACO was one of the two mainstream paradigms of early swarm-intelligence research [12].

3.1.1.3 Pheromone Update Rules and Heuristics

In standard ACO on graphs, a probabilistic transition rule weights the pheromone intensity τ and the heuristic desirability η; pheromone evaporation and a deposition proportional to

the quality of the tour are then applied:

$$p_{ij}^{k}(t) = \begin{cases} \dfrac{\tau_{ij}(t)^{\alpha}\,\eta_{ij}^{\beta}}{\sum_{u\in\mathcal{N}_i^k}\tau_{iu}(t)^{\alpha}\,\eta_{iu}^{\beta}}, & j \in \mathcal{N}_i^k, \\ 0, & \text{otherwise}, \end{cases} \tag{3.1}$$

$$\eta_{ij} = \frac{1}{d_{ij}}, \tag{3.2}$$

$$\tau_{ij}(t+1) = (1-\rho)\,\tau_{ij}(t) + \sum_{k=1}^{m}\Delta\tau_{ij}^{k}, \tag{3.3}$$

$$\Delta\tau_{ij}^{k} = \begin{cases} \dfrac{Q}{L_k}, & \text{if } (i,j) \text{ is in the tour of ant } k, \\ 0, & \text{otherwise.} \end{cases} \tag{3.4}$$

In ACS, a local update that decreases the pheromone after each move lowers the pheromone on recently used edges, which encourages exploration and prevents premature stagnation [55].

3.1.1.4 Theoretical Aspects and Runtime Analysis

- **Construction-graph model:** ACO can be modeled as the construction of solutions on a graph with restricted transition neighborhoods and stochastic selection, which allows probabilistic runtime bounds to be derived, for example through fitness-level arguments [62].
- **Convergence:** Formal results show that the pheromones are bounded and that, with sufficient iterations and suitable parameter ranges, the probability of sampling optimal tours is positive; probabilistic convergence is thereby established for the basic ACO models [62].
- **Controller tuning:** Grid-based ACO has been used to tune the parameters of nonlinear PID (NLPID) controllers, and its convergence has been demonstrated in simulation loops for mechatronic systems, which illustrates the synergy between control and optimization [55]. The corresponding block diagram is shown in Figure 3.2.

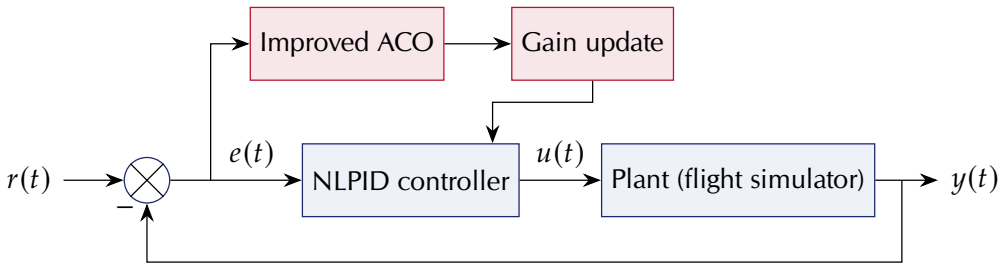


Figure 3.2. Using ACO to tune the parameters of an NLPID controller (redrawn after [55]).

3.1.2 Particle Swarm Optimization (PSO)

PSO models a swarm of particles that update their velocities and positions by combining inertial motion, their personal best experience, and the social best, yielding a simple yet powerful optimizer with strong empirical convergence and low computational overhead [6].

Compared with many heuristics, PSO requires few parameters, is inherently parallelizable, and is effective for continuous trajectory parameterization, controller tuning, formation shaping, and multi-robot task allocation [63].

3.1.2.1 Inspiration from Bird Flocking and Fish Schooling

PSO was proposed by Kennedy and Eberhart in 1995 as a model of the social behavior of bird flocks and fish schools [26]. Each particle is a candidate solution in the search space, and its motion combines its current velocity, the best position it has found itself (the personal best p_i), and the best position found by the swarm or its neighborhood (the global best g). This cognitive–social mechanism reflects alignment and cohesion in flocks and schools, while inertia supports exploration [6], [64]:

$$v_i^{t+1} = w\, v_i^t + c_1 r_1 \left(p_i - x_i^t\right) + c_2 r_2 \left(g - x_i^t\right), \qquad x_i^{t+1} = x_i^t + v_i^{t+1}, \qquad r_1, r_2 \sim U(0,1). \quad (3.5)$$

3.1.2.2 Variants and Improvements (IPSO-DAI, NisB PSO)

- **Parameter scheduling:** Linearly decreasing the inertia weight w improves global exploration early in the search and local exploitation later on, reducing the risk of premature convergence in complex landscapes [63].
- **PSO with integral control (IPSO-DAI):** Integral-controlled PSO (ICPSO) adds two integral controllers to the update to keep the population diverse; IPSO-DAI extends it with dispersed accelerator information, so that a predicted velocity index decides whether a particle moves in a divergent or a convergent manner. On high-dimensional multimodal benchmarks with many local optima, it outperformed standard PSO and ICPSO [55].
- **Topology:** Ring topologies (lbest PSO) restrict the social neighborhood, and niching splits the swarm into sub-swarms (e.g., kPSO and NichePSO) that maintain multiple optima for multimodal path planning and multi-objective missions [55]; the robotic Darwinian PSO (RDPSO) uses the same idea of smaller sub-swarms to escape local optima with real robots [1].
- **Hybrids:** Combining PSO with B-spline smoothing, gravitational search, or symbiotic search improves compliance with kinematic/dynamic constraints and enhances the smoothness of solutions for flight and driving paths.
- **Oscillating and collision-free PSO:** Adding second-order/oscillatory terms together with collision-free constraints keeps the particles inside the feasible region in constrained navigation domains.

3.1.2.3 Multi-Objective PSO (MOPSO)

MOPSO generalizes single-objective PSO by replacing the single global best g with archives of non-dominated solutions and with crowding or clustering criteria that maintain a diverse Pareto front; it also redefines how the guides for p_i and for the group are selected, since with conflicting objectives there is no single best position [55]. Representative variants include:

- **Adaptive MOPSO (AMOPSO):** The inertia and acceleration coefficients are adapted over the iterations.
- **Constrained adaptive MOPSO (CAMOPSO):** A penalty-based mechanism lets marginally infeasible solutions return to the feasible region, which helps when the feasible space is disconnected.
- **Non-dominated sorting PSO (NSPSO):** Non-dominated sorting is used to rank the swarm and manage the front.
- **Clustering:** Clustering bounds the size of the archive and keeps the stored Pareto set well distributed.

A MOPSO generates a whole set of non-dominated solutions in one run, after which fuzzy decision-making can pick the best compromise [55].

3.1.2.4 Application to UAV Swarm Path Planning

- **Optimal paths:** PSO plans short and safe paths for multiple vehicles while respecting obstacle and mission-time constraints; PSO formulations for simultaneous arrival and for minimizing mission time are common in multi-UAV scenarios [65]. An example is shown in Figure 3.3.
- **Distributed control and reconnaissance:** PSO updates have been used directly as the position commands of real quadcopters in swarm search [66]; cooperative PSO generates safe, flyable paths for multi-UAV rendezvous that respect kinematic limits and are often smoothed with B-splines for curvature continuity [14].
- **Controller and parameter tuning:** Two-layer PSO frameworks tune the parameters of an inner target-tracking loop, while an outer loop optimizes the inner gains, enabling rapid gain synthesis across many geometric paths.
- **Benchmarking:** PSO is a standard baseline for comparison with newer bio-inspired methods (such as pigeon-inspired optimization) in aerial planning [14], [37].

3.1.3 Bee-Inspired Algorithms

Bee-inspired approaches mimic the division of labor and the recruitment of foragers through waggle-dance-like communication in order to balance exploration (scouts) and exploitation (employed and onlooker bees), producing robust global search dynamics with few hyperparameters [67].

Honey-bee-based frameworks also include honey bee mating optimization (HBMO), the Bees Algorithm (BA), and bee-colony-inspired methods tailored to discrete constraints (e.g., time windows) in logistics [67].

3.1.3.1 Honey Bees and the Artificial Bee Colony (ABC)

ABC [68] divides the agents into employed bees (attached to food sources, i.e., the current solutions), onlookers (which select sources probabilistically according to their fitness), and scouts (which perform random exploration or re-initialization after an abandonment limit is reached); with the colony size and the number of cycles as its main control parameters, it delivers robust performance [55].

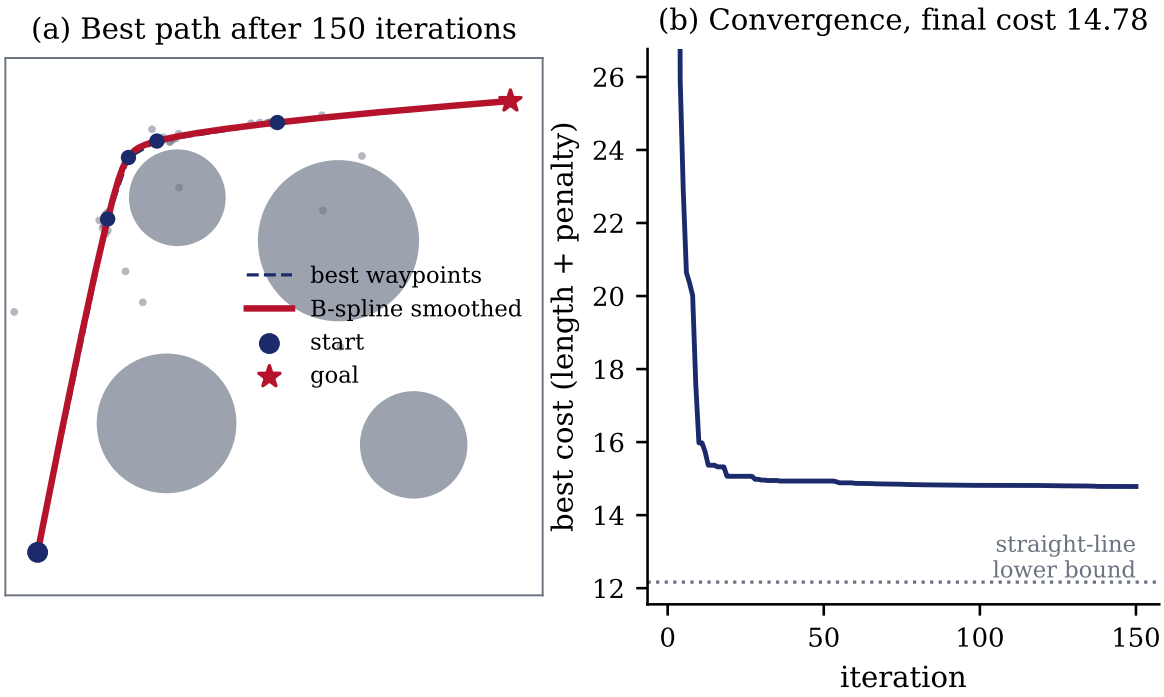


Figure 3.3. PSO path planning: each particle encodes four waypoints between the start and the goal, and the cost is the path length plus a penalty for entering an inflated obstacle. (a) The best waypoints after 150 iterations, smoothed with a cubic B-spline; the grey dots are the final positions of the swarm. (b) Convergence of the best cost.

Its advantages include simplicity, robustness, and a small set of parameters, although the search can slow down near the optimum and diversity can decrease. Variants such as BCiA (for the vehicle routing problem with time windows, VRP-TW) and BA (for training neural networks) adjust the balance between exploration and exploitation and the problem encoding for discrete settings [55].

3.1.3.2 Hierarchical and Multi-Island ABC Models

Parallel and distributed ABC variants, by offloading evaluations, sharing elite solutions, and co-evolving subpopulations, improve the scalability of high-dimensional path planning, reducing the wall-clock time while preserving diversity [55].

- **Master–worker ABC:** Central coordination parallelizes costly evaluations and local searches on worker nodes to accelerate time-critical planning [55].
- **Multi-hive (multi-island) ABC:** Independent hives periodically migrate elite solutions and, through co-evolution, raise the quality of the solutions while containing the growth of computation [55].
- **Hierarchical–hybrid ABC (HH-ABC):** Islands exchange elites at the upper level, and each island internally runs a master–worker ABC, combining diversification with parallel speedup at large scale [55].

In comparisons with an enhanced sequential ABC that also uses local search, the parallel and multi-population models consistently reached good solutions with less computation on problems with many variables [55].

3.1.4 Other Swarm Intelligence Algorithms

Many other natural metaphors have produced metaheuristics that are used for high-dimensional planning and optimization in swarm robotics [5], [6].

3.1.4.1 Firefly, Flower Pollination, and Artificial Flora Algorithms

In the firefly algorithm (FA), the brightness of a firefly encodes the quality of its solution, and a dimmer firefly moves toward a brighter one, so the population gathers around the best individual [69]. A common form of the update is

$$x_i \leftarrow x_i + \beta_0\, e^{-\gamma r_{ij}^2}\left(x_j - x_i\right) + \alpha\, \epsilon_i, \tag{3.6}$$

where r_{ij} is the distance between fireflies i and j, β_0 is the attractiveness at zero distance, γ is the light absorption coefficient, and the random term $\alpha\epsilon_i$ keeps the search from stagnating. The flower pollination algorithm (FPA) imitates the global and local pollination of flowering plants [70], and the related artificial flora (AF) algorithm imitates how plants spread seeds over an area to find the best place to grow, generating "offspring plants" around an "original plant" with propagation functions; in its authors' comparisons, AF was more stable and accurate than PSO and ABC [71].

3.1.4.2 Fireworks and Dragonfly Algorithms

The fireworks algorithm (FWA), introduced by Tan, models the explosion of fireworks: good solutions act as explosion sites around which "sparks" (new candidate solutions) are generated to refine the search [72]. The guided fireworks algorithm reappears in Section 3.4 as the optimizer of a learned search policy. The dragonfly algorithm imitates the static and dynamic swarming of dragonflies [73], and its strategy parameters have been tuned to make multi-target search by robot swarms more efficient [24].

3.1.4.3 Glowworm Swarm Optimization (GSO)

GSO is designed for multimodal problems in which several optima, or several sources, must be found at once. Each glowworm carries a luminescence quantity, *luciferin*, which grows with the fitness J at its position and decays over time,

$$\ell_i(t+1) = (1-\rho)\,\ell_i(t) + \gamma\, J\big(x_i(t+1)\big), \tag{3.7}$$

and each agent moves toward a brighter neighbor within an adaptive decision range. Because distant individuals have no influence, the swarm splits automatically into sub-swarms around the different optima. GSO has been used for localizing multiple odor sources and tracking mobile signal sources with groups of robots [55], [74].

3.1.4.4 Distributed Hybrid Fish Swarm Optimization (DHFSOA)

DHFSOA is an example of the trend toward specialized, hybrid swarm algorithms. Based on the collective search of fish schools, it was proposed for event-driven sensor deployment in underwater acoustic sensor networks, where conventional redeployment algorithms become complicated, and it finds good deployments faster than traditional methods [75].

3.2 Graph-Based and Sampling-Based Methods

Graph- and sampling-based planners convert continuous motion planning into a tractable discrete search by building graphs or trees in the free space; this approach is particularly effective in high-dimensional or geometrically complex environments, where cell grids or potential fields struggle [15].

These methods offer probabilistic completeness or strong practical performance while remaining compatible with the decentralized execution that is common in swarms [14].

3.2.1 Probabilistic Roadmap (PRM)

PRM is a classical multi-query sampling-based planner that builds a roadmap of sampled collision-free configurations and connects them with local planners; it then links the start and goal points to the roadmap and finds a path through graph search [76]. Its multi-query nature and relatively low query cost have made PRM a common backbone that is combined with metaheuristics or behavioral rules in multi-robot systems and swarms [14], [77]. Complete planners, which find a path or prove that none exists, need time that grows exponentially with the number of degrees of freedom, which is why randomized methods such as PRM are preferred in high dimensions [76].

3.2.1.1 Roadmap Construction in High-Dimensional Spaces

PRM tames the exponential growth of the configuration space with increasing degrees of freedom by random sampling, approximating the connectivity of C_{free} without an exhaustive search. The pipeline has two phases: a preprocessing phase, which builds the roadmap by sampling collision-free milestones and connecting nearby neighbors with a simple local planner, and a query phase, which connects the start and goal to the roadmap and runs a graph search for a feasible path [77].

The roadmap approximates a graph over C_{free} whose vertices are the milestones and whose edges are feasible local connections; its completeness is probabilistic and improves with the number of samples and the connection radius [77].

- **Managing complexity:** Performance depends on the number and placement of the milestones; informed samplers such as Medial-Axis PRM (MAPRM) bias the samples toward the medial axis to increase safety margins, yielding higher-quality roadmaps with fewer samples [4].

3.2.1.2 Integration with Flocking and Shepherding

Roadmaps inject a global navigation structure into swarms that interact mainly locally, enabling coordinated homing, exploration and coverage, passage through narrow corridors, and shepherding strategies.[1] Flocks can traverse the edges of the roadmap while node-level rules for separation, alignment, and cohesion preserve the formation and spacing, especially in narrow regions or bottlenecks [4]. Roadmaps have supported several group behaviors:

- **Homing and goal searching:** The roadmap gives the flock the structure it needs to reach a known goal or to search for an unknown one.

[1] Shepherding: an external agent selects a path on the roadmap and, by issuing commands, guides the swarm along it, so that global guidance is blended with the local cohesion of the flock [4].

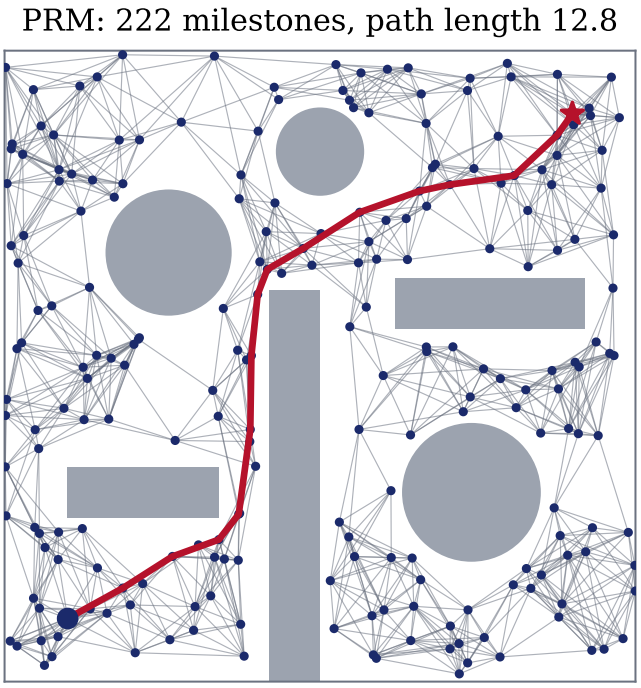


Figure 3.4. PRM in a cluttered workspace: collision-free milestones are connected to their ten nearest neighbours, and Dijkstra's algorithm finds the path on the roadmap.

- **Exploration and coverage:** The edges gain weight as robots traverse them, and at each junction the flock chooses probabilistically in favor of lower-weight edges, so less-visited corridors get priority.
- **Narrow passages:** Roadmap nodes can carry rules that the flock members apply when they reach them, which coordinates the flock's configuration in narrow or complex areas.
- **Shepherding:** An external agent extracts a path from the roadmap and steers the flock from behind along it.

3.2.2 Rapidly-Exploring Random Trees (RRT)

RRT samples the state space and greedily extends a tree from the nearest node toward each sample, rapidly exploring unknown regions and quickly producing feasible (though not necessarily optimal) paths [15], [78].

RRT is probabilistically complete and accommodates kinodynamic constraints, which makes it practical for robots with dynamics and control limitations.

3.2.2.1 RRT and RRT-Connect for Single-Agent Queries

RRT views motion planning as a tree search that expands toward uniform samples and connects new states to their nearest neighbor using a local steering function subject to the constraints [78].

RRT-Connect grows two trees, one from the start and one from the goal, and aggressively attempts to join them with its CONNECT heuristic, which reduces the search time compared with a single tree [79]. RRT* adds a rewiring step that makes the solution asymptotically optimal, at a higher computational cost [80]. The difference between RRT and RRT-Connect can be seen in Figure 3.5.

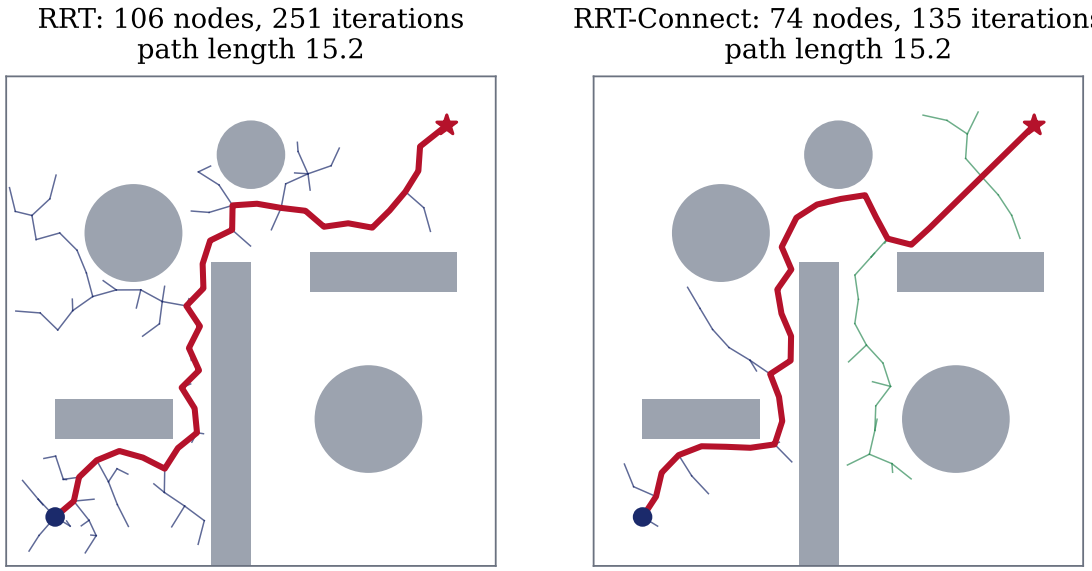


Figure 3.5. RRT and RRT-Connect in the workspace of Figure 3.4, with the same step size and random seed. RRT-Connect grows a tree from each end and joins them with its CONNECT step, so it needs fewer iterations.

3.2.2.2 Adaptations for Swarm Robotics (e.g., Virtual Tubes)

Distributed RRT enables decentralized collision avoidance: each agent plans while accounting for the other agents and the static obstacles, using local communication when needed [14].

In dynamic environments, *grafting*—inserting new intermediate waypoints into an existing RRT path when the obstacles change—avoids complete replanning and enables real-time updates for multi-robot teams [23].

- **Optimal virtual tube planning (OVTP) and Tube-RRT*:** A hierarchical approach first generates homotopic paths (e.g., with RRT*) and then builds an "optimal virtual tube" that bounds safe, obstacle-free corridors for the synthesis of a central path and for distributed execution; Tube-RRT* trades off path length against the cross-sectional area of the corridor in order to contain the swarm [16], [17].

The computational complexity of tube planning is

$$O\big(n(\log n + 1) + l\big), \tag{3.8}$$

for l homotopic paths on n samples, compared with

$$O\big(n\, l\, (\log n + 1)\big) \tag{3.9}$$

when RRT* is simply run l times [17].

3.2.2.3 Sub-Optimal Algorithms for Higher Speed

- **Multi-tree structures:** Bidirectional RRT-Connect and three-tree approaches parallelize exploration from multiple fronts and improve the success rate in narrow passages [15], [81].
- **Biased sampling:** Goal biasing accelerates convergence; obstacle-biased methods (e.g.,

Dangerzone RRT) steer the samples toward the boundaries of obstacles so that narrow corridors are traversed efficiently.

- **Roadmap-guided sampling:** GVD-guided RRT uses the generalized Voronoi diagram (GVD) as a heuristic backbone and greatly accelerates planning in maze-like scenes.
- **Dynamic replanning:** DRRT prunes and repairs only the invalidated parts of the path when changes occur; ERRT resumes growth when the path is blocked; and multipartite RRTs (MPRRT) combine fast pruning with restart behavior to maintain responsiveness [13].
- **Pseudo-random trees:** The multi-agent rapidly-exploring pseudo-random tree replaces random sampling with a deterministic sequence, keeping probabilistic completeness while converging quickly [77].

3.2.3 The A* Algorithm and Grid-Based Methods

Deterministic graph and grid planners are fundamental for structured maps: A* yields optimal paths with the help of heuristics, and grids and cell decompositions provide discrete models that are suited to proofs of correctness and to integration with other planners. These methods are often used as baselines or as components of hybrid multi-robot path planning systems [14], [15].

3.2.3.1 A* for Optimal Paths in Static Environments

A* evaluates nodes with $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start and $h(n)$ is an admissible heuristic estimate of the cost to the goal; this guarantees completeness and optimality on graphs with nonnegative edge weights [82]. An example of how this method works is shown in Figure 3.6; its weakness is that the search effort still grows quickly as the search space expands [15].

With informed heuristics, A* is more efficient than Dijkstra's algorithm in large spaces, and it is often combined with fuzzy neural networks to speed up task completion and improve controllability and adaptability in multi-robot systems [37].

3.2.3.2 Grid-Based Environment Representation

Grid maps discretize space into cells, turning continuous motion into a graph search for algorithms such as A*, D*, or even ACO on grids, and they can be updated online to handle dynamics [15]. The resolution is a critical trade-off between accuracy and computation or memory: a very fine grid is expensive and a very coarse one is inaccurate, which is why multi-resolution or hierarchical grids are attractive in practice [14].

3.2.3.3 Cell Decomposition Techniques (e.g., Boustrophedon, Morse)

Cell decomposition partitions C_{free} into non-overlapping cells and builds an adjacency graph whose nodes are the cells and whose edges are adjacency relations; optimal paths are obtained by a graph search across the cells with methods such as Dijkstra's algorithm or A* [15].

Boustrophedon decomposition supports exact coverage by sweeping and splitting the space at points where its connectivity changes [83], whereas Morse decompositions use

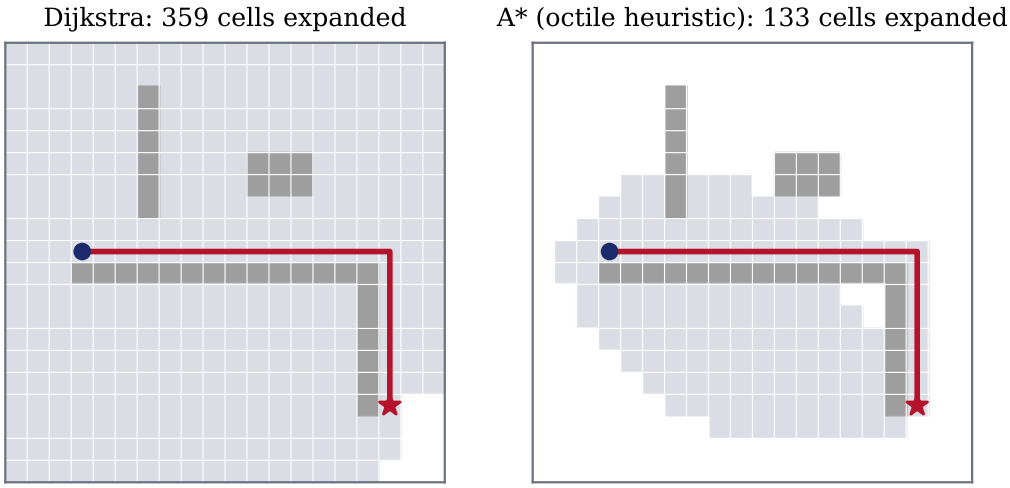


Figure 3.6. Dijkstra's algorithm and A* on the same 8-connected grid. Both return the same optimal path, but the admissible octile heuristic lets A* expand far fewer cells.

the critical points of a Morse function to produce cells that are easily covered with simple back-and-forth patterns once entered [84].

3.2.3.4 Generalized Voronoi Diagrams (GVD) for Narrow Regions

The GVD partitions the free space into loci that are equidistant from the obstacles: its edges are the points equidistant from two obstacles, and its meet points are equidistant from three or more. By maximizing the safety margin, these edges and junctions serve as a roadmap for safe passage through cluttered environments and narrow passages [84], [85].

Planning can move along the GVD, reach its edges near the goal, and then exit toward the goal. The GVD also guides sensor-based coverage, accelerates RRT in maze-like environments, and supports path-priority strategies for multiple robots in narrow passages [85].

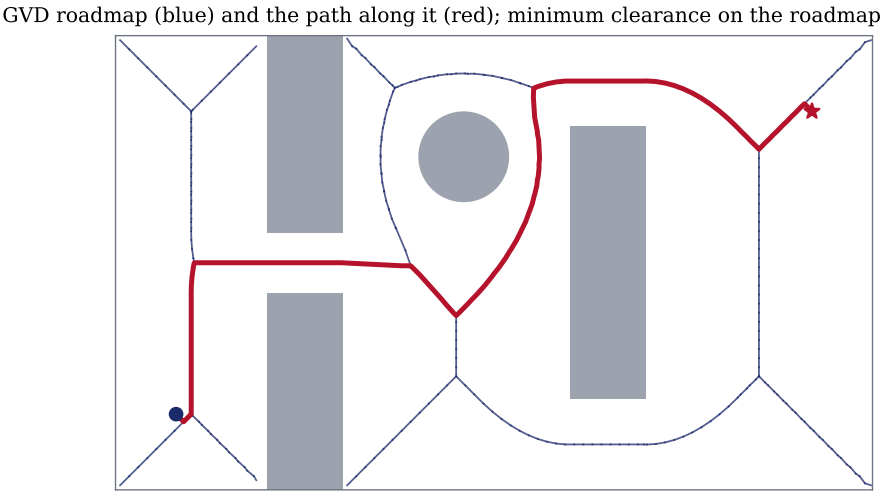


Figure 3.7. An approximate generalized Voronoi diagram, built from the ordinary Voronoi diagram of points sampled on the obstacle boundaries. The path follows the roadmap through the narrow door and keeps the largest possible distance from the walls.

3.3 Potential Field Methods

Potential field planners, and particularly the artificial potential field (APF), turn path planning into gradient-following motion by defining deterministic, rule-based potentials. Their simplicity, fast computation, and real-time reactivity in dynamic scenes make them attractive for distributed swarms [86].

In multi-robot systems, the APF is usually employed as a local planner or control law that integrates smoothly with higher-level graph or sampling planners or with metaheuristics, keeping the computational cost per agent low while preserving decentralized execution [87].

3.3.1 Artificial Potential Fields (APF)

The APF, pioneered by Khatib in 1986, remains the standard for geometric navigation based on force fields: the goal creates attraction in the configuration space, the obstacles create repulsion, and the motion commands are derived from the negative gradient of the potential [88].

In multi-agent settings, the APF models inter-agent and agent–environment interactions as attractive–repulsive potentials, and gradient-based controls ensure formation, tracking, and collision avoidance with decentralized feedback [89].

3.3.1.1 Attractive and Repulsive Forces for Navigation

The APF defines a total potential $U(X)$ as the sum of attractive and repulsive components and computes the net control force as the negative gradient of this potential, so that the robot descends toward the goal while keeping away from the obstacles:

$$U(X) = U_{\text{att}}(X) + U_{\text{rep}}(X), \qquad F(X) = -\nabla U(X) = F_{\text{att}}(X) + F_{\text{rep}}(X). \tag{3.10}$$

Here, U_{att} pulls the agent toward the goal, and U_{rep} increases sharply near the obstacles to preserve safety, providing smooth closed-loop motion that is suitable for real-time embedded control [87]. A common attractive potential is quadratic, $U_{\text{att}}(X) = \frac{1}{2} k_{\text{att}} \|X - X_g\|^2$, and a typical repulsive potential is

$$U_{\text{rep}}(X) = \begin{cases} \dfrac{1}{2}\, k_{\text{rep}} \left(\dfrac{1}{d(X)} - \dfrac{1}{d_0} \right)^2, & d(X) \le d_0, \\ 0, & d(X) > d_0. \end{cases} \tag{3.11}$$

Consequently, $\|\nabla U_{\text{rep}}\|$ grows without bound as $d(X) \to 0$, which gives priority to collision avoidance near the boundaries [90].

3.3.1.2 Principles of Obstacle Avoidance and Goal Seeking

- **Goal seeking:** $U_{\text{att}}(X)$ is shaped so that the magnitude of the attractive force is large far from the goal and zero at the goal; this ensures natural deceleration and stopping at the potential minimum and encodes precise capture of the goal in static scenes [87].
- **Obstacle avoidance:** $U_{\text{rep}}(X)$ grows rapidly as the distance to the obstacles decreases and can vanish outside a radius of influence; this design enforces separation because the repulsive gradient dominates near the boundaries [89].

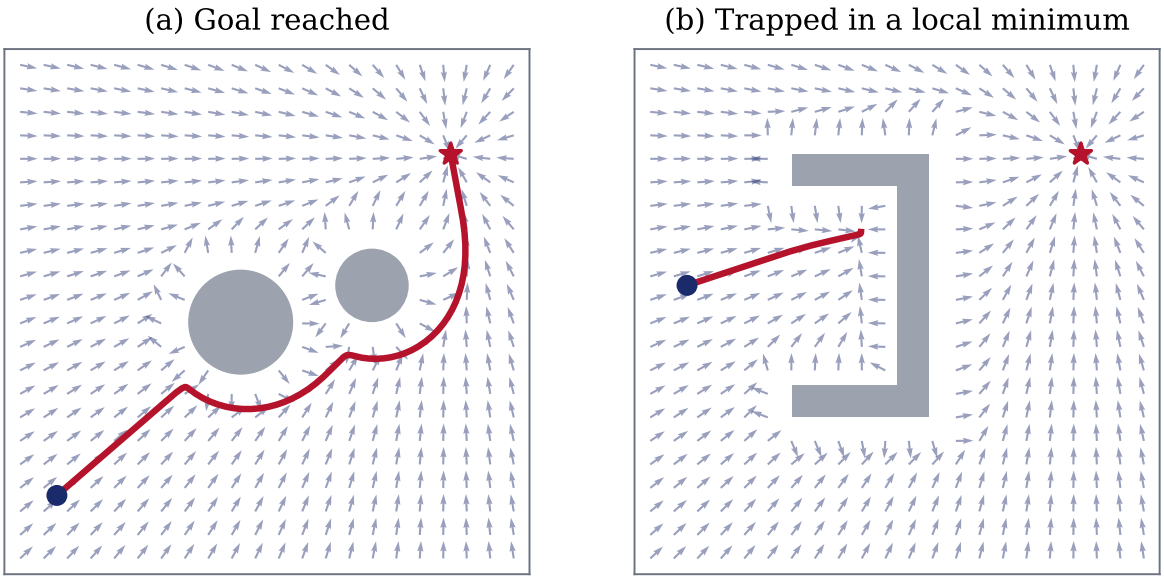


Figure 3.8. The force-field view of robot path planning: the arrows show the direction of $F = -\nabla U$. (a) The target attracts and the obstacles repel the robot, which reaches the goal. (b) A U-shaped obstacle creates a local minimum in which the same controller gets stuck.

- **Known limitations:** The classical APF may suffer from local minima and is sensitive to parameter tuning, which can trap the robot away from the global minimum; hybridization or adaptive shaping is commonly used in practice to mitigate this problem [90].

3.3.1.3 Improved Potential Field for Cooperative Target Hunting

To overcome local minima and embed multi-robot cooperation, improved potential fields (IPFs) add cooperation measures and problem-specific shaping, which enables coordinated target hunting by multi-AUV teams in turbulent, dynamic surface-water environments [41]. The cooperation is expressed through three measures: a dispersion degree H_D that spreads the agents apart, a homodromous degree H_H that keeps their headings consistent, and a district-difference degree H_{DD} that distributes the search; a term in the relative distance to the target makes sure that the only global minimum is at the target, which removes the local-minimum problem [41].

A representative formulation of the total potential for AUV R_i is the weighted combination

$$U_i = \omega_\alpha U_\alpha + \omega_\gamma U_\gamma, \tag{3.12}$$

where the attractive component U_α covers the exploration and target-tracking phases, and the repulsive component U_γ combines cooperation degrees for dispersion, heading alignment (homodromy), and district difference (spatial distribution) with the density weights $\omega_{\gamma D}$, $\omega_{\gamma H}$, and $\omega_{\gamma DD}$ [41].

The attractive component is defined piecewise as

$$U_\alpha\left(X_{R_i}\right) = \begin{cases} \dfrac{1}{2} k_1 \Phi\left(\dfrac{N_u}{N}, \dfrac{A}{\bar{A}}\right), & \text{exploration,} \\ \dfrac{1}{2} k_2 \left\| X_{R_i} - X_{T_l} \right\|^2, & \text{moving to the target,} \end{cases} \tag{3.13}$$

where Φ increases with the proportions of undetected targets and unexplored areas, and the target-tracking term uses a quadratic pull toward the assigned target T_l [41].

The repulsive and cooperative component is defined as

$$U_\gamma = \begin{cases} \omega_{\gamma D} H_D + \omega_{\gamma H} H_H + \omega_{\gamma DD} H_{DD}, & \text{exploration}, \\ \dfrac{1}{2}\kappa\left(\dfrac{1}{d} - \dfrac{1}{d_1}\right)^2 + (\text{cooperation terms}), & \text{near obstacles}, \end{cases} \tag{3.14}$$

where H_D encourages spacing, H_H increases the coherence of headings, and H_{DD} promotes spatial distribution to prevent excessive concentration; the obstacle term explicitly maintains the safety margin [41].

Performance evidence from Monte Carlo experiments with six AUVs, two dynamic targets, and obstacles in a 120×120 m^2 area shows that the IPF reduces the path length and the hunting time compared with the classical potential field, indicating greater efficiency, robustness, and cooperation even when agents fail. The IPF also maintained high success rates under random layouts and dynamics, and it outperformed baseline PSO methods in obstacle-rich hunts while preserving collision-free pursuit and distributed coordination [41].

3.4 Reinforcement Learning and Deep Learning Approaches

AI learning methods take path planning beyond fixed rules by allowing agents to optimize their behavior through experience, which makes them robust to uncertainty and environmental change in multi-robot systems. Reinforcement learning (RL) and its extensions are widely used for decentralized decision-making, real-time collision avoidance, and cooperative tasks in swarms [91].

3.4.1 Reinforcement Learning (RL)

RL formulates sequential decision-making as the optimization of the expected cumulative reward through repeated interaction with the environment, enabling data-driven improvement without labeled datasets. This adaptability makes RL effective in the dynamic, partially known, or adversarial environments that are common in real-world multi-robot deployments.

3.4.1.1 Learning Optimal Behaviors through Trial and Error

Agents learn policies $\pi(a \mid s)$ that maximize the objective

$$\max_\pi \ \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r_t\right], \qquad \gamma \in (0,1), \tag{3.15}$$

while exploration strategies ensure sufficient coverage of the state–action space during learning.

Policy optimization can be value-based, policy-gradient, or actor–critic, and it is chosen according to the complexity of the task, the observability, and the properties of the action space. The goal of policy optimization is to compute a mapping $\pi^* : S \to A$ that maximizes the long-term return under the dynamics of the environment while balancing exploration and exploitation.

- **Adaptability:** RL adapts to environmental changes by continuously updating value functions or policy parameters, maintaining performance in situations where classical planners would need costly replanning.
- **Learning without labels:** RL does not require supervised learning with labels; agents learn from numerical rewards that encode the task objectives and safety constraints.

3.4.1.2 Optimal Flight Paths for UAV Swarms

Multi-agent RL (MARL) with centralized training and decentralized execution (CTDE) supports cooperative navigation by training shared critics or common value baselines, while decentralized policies run on the individual UAVs [21], [92].

MARL policies compute efficient coverage and routing for various map sizes and swarm sizes, improving mission time and battery consumption compared with hand-crafted rules [93].

- **Cooperative navigation:** CTDE allows agents to learn coordination (e.g., collision-free formations and simultaneous arrival) while keeping run-time control decentralized. This scheme is shown in Figure 3.9.
- **Coverage optimization:** Learned policies maximize the reward for first-visited cells and minimize repeated traversal, achieving scalable area coverage [93].
- **Hybrids and speed:** Neural-network function approximation accelerates Q-learning; a global neural network and heuristic Q-learning variants speed up convergence and reduce the hardware requirements for on-board deployment [14].

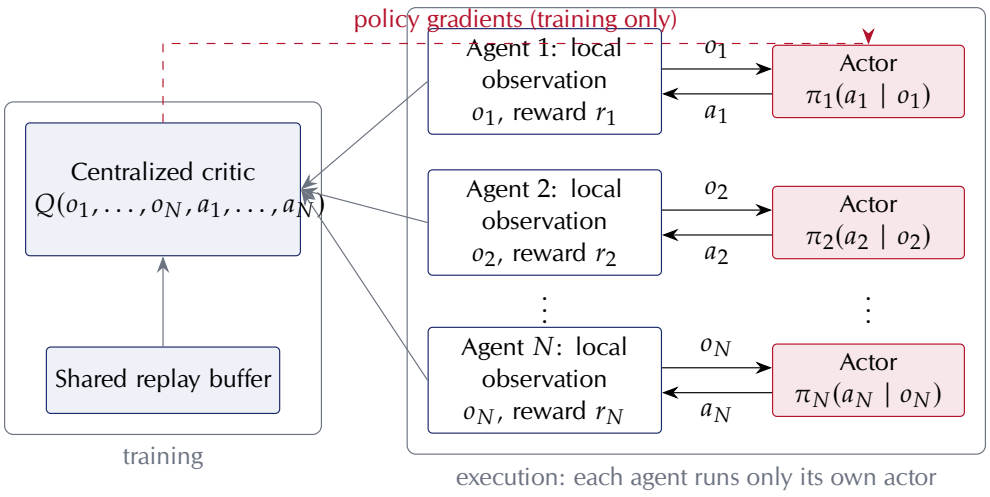


Figure 3.9. The CTDE scheme: a centralized critic is trained on the observations and actions of all agents, while each agent executes its own decentralized actor.

3.4.1.3 Rewards and Interaction with the Environment

Reward design directly steers the learning process and must encode safety, efficiency, and cooperation for swarms. In coverage tasks, first-time visits usually receive larger rewards, collisions, oscillations, and deadlocks are penalized, and shaping terms are added to speed up learning.

- **Incentive mechanism:** Positive rewards for progress toward the goal and for coverage, and negative rewards for collisions or constraint violations, lead to a stable improvement of the policy.
- **Credit assignment:** Global team rewards can mask individual contributions ("lazy agents") and make credit assignment difficult in swarms.
- **Solutions:** Local rewards, heterogeneous reward components, and difference rewards or agent-specific critics factor global objectives into per-agent feedback, improving the stability and fairness of learning.

3.4.2 Deep Reinforcement Learning (DRL)

DRL, by integrating deep neural networks with RL, handles the high-dimensional observations, partial observability, and continuous control that are common in robotics and swarms. DRL reduces the reliance on static maps by learning from raw or learned features, and it supports real-time inference on embedded accelerators [94].

3.4.2.1 Path Planning in Complex and Dynamic Environments

DRL scales to dynamic obstacles and narrow passages, where classical methods face slow convergence or limited representational capacity [94].

Typical progressions include the deep Q-network (DQN), whose experience replay improved data efficiency [95], and Double DQN for greater stability, followed by actor–critic methods for continuous actions, such as DDPG and TD3, which adds double Q-learning to DDPG [94], [96].

- **High dimensionality:** DRL learns collision avoidance and multi-agent coordination directly from sensor and state embeddings, without hand-crafted features or complete global maps [19].
- **Limits of value-based approaches:** DQN and Double DQN struggle in continuous action spaces; policy-gradient methods (DDPG, TD3) mitigate this problem, but TD3 relies on injected noise and may under-explore in complex terrains [94].

3.4.2.2 Soft Actor-Critic (SAC) Algorithms for Continuous Actions

SAC promotes robust exploration in continuous control by maximizing a maximum-entropy objective, jointly optimizing the expected return and the entropy of the policy [94], [97]:

$$J(\pi) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t \Big(r_t + \alpha\, \mathcal{H}\big(\pi(\cdot \mid s_t)\big)\Big)\right], \qquad \alpha > 0. \tag{3.16}$$

- **Exploration and robustness:** Entropy regularization reduces premature convergence to sub-optimal behaviors and improves performance under non-stationary conditions.
- **Advanced variants (e.g., GAP_SAC):** GAP_SAC combines a dynamic heuristic reward inspired by heuristic search, which gives a smooth signal toward the goal, with prioritized experience replay and a gated attention mechanism that focuses the network on the important features of the environment. In dynamic and narrow scenarios it produced shorter paths and converged faster than SAC and TD3 [94].

- **Prioritized replay:** Transitions are prioritized according to their temporal-difference (TD) error, and the resulting replay bias is corrected through importance sampling [98]:

$$P(i) \propto |\delta_i|^{\alpha} . \tag{3.17}$$

- **State augmentation:** Adding features such as the Euclidean distance to the goal (DG) and the amount of heading correction (MA) improves observability and planning performance for mobile robots [94].

3.4.3 Imitation Learning (IL)

Imitation learning (IL), or learning from demonstration, learns policies from expert trajectories: instead of optimizing an unknown reward, it minimizes the discrepancy between actions, thereby reducing the cost of exploration and accelerating convergence in large, complex spaces. IL is well suited to multi-robot domains in which accurate experts (humans or centralized controllers) are available but online trial and error is costly or unsafe [22].

3.4.3.1 Learning from Expert Demonstrations

Given expert trajectories $\tau_i = \{(o_t, a_t)\}_{t=1}^{T}$, IL builds a dataset $D = \{(s_k, a_k)\}_{k=1}^{N}$ and trains a policy network by minimizing losses such as the mean squared error (for continuous actions) or the cross-entropy (for discrete actions) between the predicted and the expert actions [22].

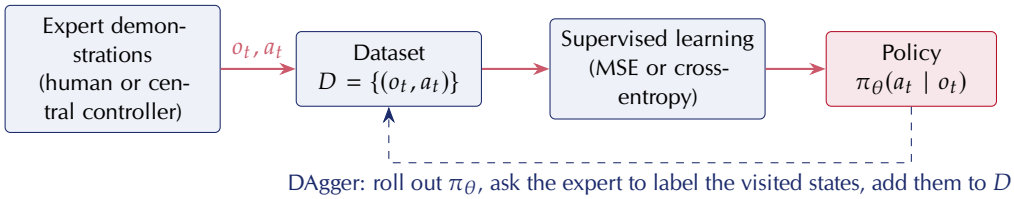


Figure 3.10. The imitation learning approach: expert demonstrations form a training dataset from which a policy $\pi_\theta(a_t \mid o_t)$ is learned by supervised learning.

In Figure 3.10, the expert takes action a_t given the observation o_t; the pairs (o_t, a_t) form the dataset D on which the policy is trained.

In swarms, behavior cloning combined with graph neural encoders can distill centralized experts (e.g., APF-based flocking) into decentralized policies that are compatible with the constraints of local communication, with performance close to that of the centralized expert [99].

- **Advantages:** IL bypasses exploration under sparse rewards and produces strong initial policies that can be refined with RL if needed.
- **DAgger:** Dataset aggregation iteratively queries the expert on the states visited by the current policy and expands D, improving generalization to out-of-distribution states [99], [100].

3.4.3.2 A Two-Stage Framework for Multi-Target Search

A two-stage framework first uses deep learning to imitate a high-performing target strategy from behavioral data and then refines the weights of the learned network through evolutionary optimization to tune its performance in multi-target search [22]. It is meant for the case in

which a good "target strategy" can be observed through behavioral data and performance figures but its implementation is unknown; in the reported study, a search strategy based on probabilistic finite state machines played that role.

For example, after a ReLU-based network with dropout (RNS) has been trained, an evolutionary optimizer such as the Guided Fireworks Algorithm (GFWA) refines its parameters to minimize metrics such as the mean number of iterations (mI) needed to find and collect all targets, yielding a final strategy (e.g., RNSE) whose efficiency and stability are close to those of the target strategy [22].

- **Integration:** This approach captures complex collective behaviors from demonstrations and then uses evolutionary search to escape local minima in the parameter space [22].
- **Outcome:** Reports show that the optimized IL strategy achieves high success rates with stable convergence across random layouts and is comparable or superior to the target strategy under swarm constraints [22].

3.5 Hybrid Approaches

By integrating complementary algorithms, hybrid methods prevent premature convergence, exploit the structure of the problem, and scale to large spaces with dynamic obstacles. A common pattern pairs fast, consistent global planners with adaptive optimizers or learned policies for responsiveness and robustness in swarms [14].

Hybrids are usually organized as either *cooperative* (separate modules that exchange information) or *embedded* (one method nested inside another), and they are chosen to optimize metrics such as duration, path length, curvature/smoothness, and real-time feasibility under constraints [15], [71].

3.5.1 Integrating Multiple Path Planning Strategies

Large search spaces, nonholonomic constraints, moving obstacles, and local minima motivate combining deterministic structure with AI-based exploration or learning, in order to sustain performance at the scale of decentralized teams [14].

Cooperative hybrids exchange guidance, costs, or partial plans between modules; embedded hybrids nest optimization or learning inside a deterministic core to adapt parameters online or reshape the costs, which yields faster convergence and higher success rates [15].

3.5.1.1 Combining Deterministic and AI Techniques

- **Deterministic core:** Graph and grid planners, cell decomposition, and sampling guarantee feasibility—and sometimes optimality—in structured regions, producing collision-free paths through efficient graph search and local steering [14].
- **AI enhancement:** Bio-inspired and learning methods (e.g., PSO, genetic algorithms (GA), and RL or neural networks) inject exploration, adaptive parameter tuning, and data-driven policies that generalize beyond static environment models and cope with uncertainty and dynamics [15].
- **Synergy:** Deterministic layers provide initial feasible or short paths and handle the constraints; AI layers refine, smooth, reweight, or replan the path online, mitigating local minima and improving path quality and execution time under disturbances [14].

3.5.1.2 Example: FNN Combined with the A* Algorithm

A fuzzy neural network (FNN) joins the knowledge representation of fuzzy reasoning with the learning of a neural network, but on its own it can produce paths that are not the shortest, with sudden motion changes and dead zones. A two-stage grid-based algorithm therefore combines A* with the FNN to guarantee the feasibility of the shortest path while producing smoother and more controllable local motions than an FNN alone [37].

- **Typical pipeline:** A* provides a short global path at a coarse scale; the FNN is then applied on finer cells for smoothing and local adaptation, reducing dead zones and motion jumps while preserving the global optimality of the backbone path; the result was a shorter path and a faster task completion than with the FNN alone [37].

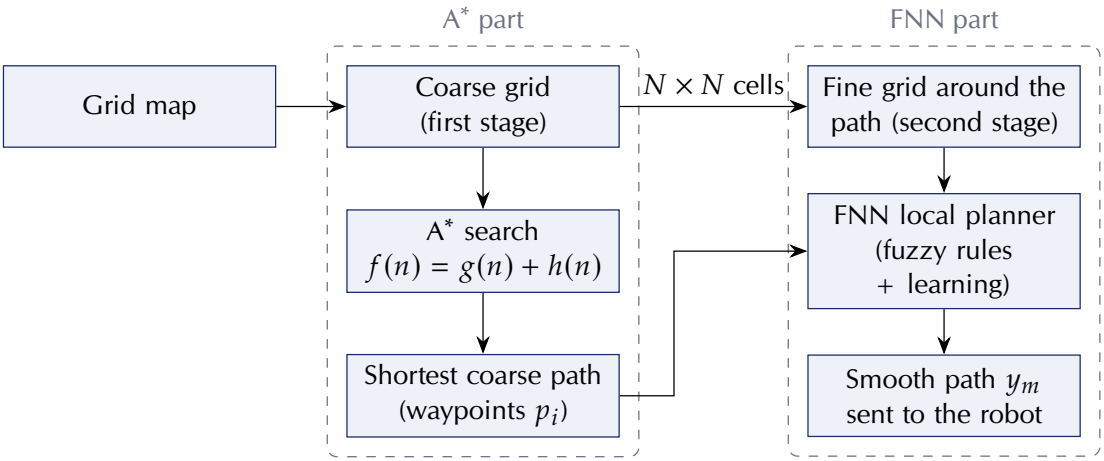


Figure 3.11. Block diagram of the FNN algorithm combined with A* (redrawn after the description in [37]).

3.5.2 Other Hybrid Patterns and Evidence

- **Deterministic–optimization hybrids:** A*+APF, A*+RL, A*+DWA, Theta*+dipole field and DWA, and sampling+optimization combinations handle dynamic obstacles and curvature limits and improve execution time in multi-robot settings while preserving smoothness [15].
- **Optimization–optimization hybrids:** WSA–APF, KA–SCA, ABC–EP, MGSA–DWA, SOMA–PSO, GWO–WOA, and DWA–TLBO show better convergence, shorter paths, fewer turns, and superior time efficiency compared with single methods on benchmark maps and dynamic scenes [15].
- **Adoption rate:** Hybrid techniques make up roughly 25% of the multi-robot path planning literature, and, because of their balance between speed and quality, further study is recommended for heterogeneous systems and dynamic targets [15].
- **Shelter seeking:** For flying swarms in 3-D, three hybrid systems (ModRIO, GaussRIO, and StdRIO) let a homogeneous swarm find shelters by following three basic instincts: random walks, staying with "good friends", and leaving the shelter. They are built to cross the flat plateaus of high-dimensional search spaces while escaping local optima [37].

3.6 Virtual Tube-Based Planning

Virtual tube methods define obstacle-free corridors that act as macroscopic "containers" for multi-robot motion, allowing global paths to be generated centrally while each agent executes in a decentralized manner inside the corridor. This idea is aligned with safe-corridor planning and with guidance based on homotopy classes, which lead groups through narrow passages while maintaining inter-robot distances and flow [16]. The idea echoes the safe flight corridors proposed for drones flying over cities, such as Airbus' SkyRoad project [16].

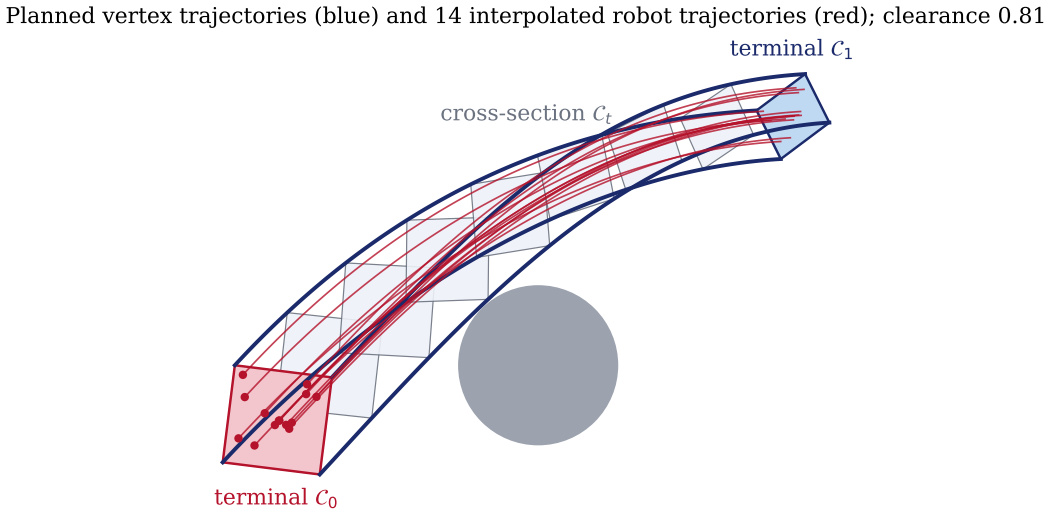


Figure 3.12. A linear virtual tube with convex-hull terminals. Only the four vertex trajectories (blue) are planned; each robot starts at a convex combination of the vertices of C_0 and follows the same combination of the vertex trajectories (red).

Figure 3.12 shows the geometric structure of a virtual tube. The start region (terminal C_0) and the goal region (terminal C_1) are the entry and exit sets of the swarm. Each robot moves along a trajectory $\mathbf{h}\big((\mathbf{q}_0, \mathbf{q}_m), t\big)$ from its start point $\mathbf{q}_0$ to its exit point $\mathbf{q}_m$, and the shaded polygons are cross-sections C_t of the tube at several times. Because every cross-section is convex and the tube avoids the obstacle, any robot that starts inside C_0 stays inside the tube and away from the obstacle, and many robots can pass through it at the same time.

3.6.1 The Concept of Virtual Tubes

A virtual tube is a bounded region of free space that connects the start and goal regions and is constructed to maximize the safety margin and maintain acceptable cross sections along the path, so that many robots can pass through simultaneously without deadlock. Once the tube is available, path planning reduces to assigning each robot a feasible path that stays inside the corridor and respects minimum distances, which greatly simplifies downstream control and coordination [16].

3.6.1.1 Definition and Components

Formally, a virtual tube T in an n-dimensional configuration space is a 4-tuple (C_0, C_1, f, h) [16]. The terminals C_0 and C_1 are disjoint, bounded, convex sets (convexity makes them simply connected); $f : C_0 \to C_1$ is a diffeomorphism that pairs every start point with an exit point, producing the ordered pairs $\mathcal{P} = \{(\mathbf{q}_0, \mathbf{q}_m) : \mathbf{q}_0 \in C_0,\ \mathbf{q}_m = f(\mathbf{q}_0)\}$; and $h : \mathcal{P} \times [0, 1] \to T$

is a smooth map that generates the trajectory of each pair. The cross-section at time t is $C_t = \{h((\mathbf{q}_0, \mathbf{q}_m), t) : (\mathbf{q}_0, \mathbf{q}_m) \in \mathcal{P}\}$, and every cross-section is simply connected and varies continuously. Earlier definitions built around a generator curve are special cases. The space is n-dimensional rather than three-dimensional because the motion of a rigid robot in 3-D is described by a six-dimensional manifold. In words:

- **Terminals:** The start and goal regions, which delimit the entry and exit sets of the swarm and are chosen as simple bounded regions so that they can be filled without conflicts.
- **Mapping and assignment:** Establishing entry–exit pairs between the entry points in the start region and the exit points in the goal region, which are often induced by a global planner that enumerates the homotopy classes around the obstacles.
- **Paths and cross sections:** A family of feasible paths inside the corridor and their instantaneous cross sections, designed to remain connected and sufficiently wide to prevent congestion and maintain separation.

3.6.1.2 Linear Tubes and the Interpolation Idea

In practice, many deployments use linear or piecewise-linear centerlines and polyhedral cross sections, so that the interior paths of the agents can be built from convex combinations of, or simple interpolations between, boundary references while preserving feasibility and spacing. A tube is *linear* when both f and h are linear maps; in the most useful case, the linear virtual tube with convex-hull terminals, C_0 and C_1 are convex hulls (polyhedra) of finite vertex sets and f maps vertex to vertex. Under the conditions given in [16], every trajectory inside such a tube that is a convex combination of optimal vertex trajectories is itself optimal. The planner therefore solves only a few boundary problems and obtains every other trajectory by interpolation, which reduces the cost from $O\big(k(n_t, \varepsilon)\, n_t^2\big)$–$O\big(k(n_t, \varepsilon)\, n_t^3\big)$ per robot to $O(n_t)$ and makes the method suitable for large swarms [16].

3.6.2 Optimal Virtual Tube Planning

OVTP selects safety-margin-aware corridors and a small set of reference paths, and it then efficiently generates the many robot paths so as to minimize congestion and computation while guaranteeing obstacle avoidance. This approach amortizes the heavy global search over the group and replaces independent global planning for each robot with a lightweight synthesis inside the tube [16].

3.6.2.1 Hierarchical Planning with MPC Tracking

A practical architecture computes the tube(s) and the reference paths centrally and then deploys distributed controllers (e.g., MPC) on each robot to track its assigned path and to enforce the tube and inter-robot constraints in real time [16].

MPC naturally handles receding-horizon collision avoidance and adherence to the boundaries through inequality constraints: the position of a robot is constrained to the interior of the tube, or to a band near its boundary for robots on the boundary. This gives formation-like cohesion and robust tracking, and in simulations and experiments the combination outperformed formation control and plain predictive control in obstacle-dense environments [16].

3.6.2.2 Objectives: Computation, Volume, and Energy

- **Computation:** By reusing the same corridor and a small set of references, the synthesis for each agent reduces to interpolation or projection and local smoothing, which lowers the complexity compared with complete replanning for every robot.
- **Minimum cross-sectional volume:** Corridors are selected to preserve a sufficient cross-sectional "capacity" at bottlenecks; margin-aware backbones such as the GVD or homotopic paths preserve a safe tube thickness. Tube-RRT* measures this by the minimum gap volume, the volume of the smallest gap the path passes through [17].
- **Energy:** Smooth curvature and non-crossing assignments reduce the demands for acceleration and turning and improve energy efficiency while maintaining throughput; choosing f so that trajectories do not intersect serves the same purpose [16].

3.6.2.3 Tube Networks for Complex Obstacles

When a single corridor is insufficient, multiple tubes form a network that splits the swarm into parallel routes around large obstacles and merges it again downstream, thereby reducing congestion and delay. Designing consistent assignments across the branches and avoiding internal crossings are essential for preserving the distinction between the boundary and the interior and for supporting safe merging behaviors; how to build such networks is still an open question [16].

3.7 Summary

This chapter reviewed and analyzed a wide range of path planning methods for swarm robotic systems. These methods were divided into three broad categories—deterministic, AI-based, and hybrid—each with its own distinctive characteristics and applications. Based on this analysis, the following conclusions can be drawn:

1. **Deterministic methods:** Classical algorithms such as A* and cell decomposition perform reliably in static, known environments and guarantee that a path will be found. However, as discussed in Section 2.4.2, these methods suffer from problems such as high computational cost and inflexibility in dynamic and complex environments. Sampling-based algorithms such as RRT* have partially addressed these challenges in high-dimensional spaces by striking a balance between speed and optimality.
2. **AI-based methods:** This category—especially bio-inspired algorithms such as PSO and ACO—is highly compatible with swarm robotics owing to its distributed nature. The review showed that PSO, through its velocity and position update formula (Equation (3.5)), enables fast convergence in dynamic environments and, in most complex scenarios, succeeds in finding the shortest path and the minimum time simultaneously. In addition, reinforcement learning methods (RL and deep Q-learning) have shown great potential for navigation in unknown environments without a prior map.
3. **Hybrid methods:** Hybrid approaches, which account for about 25% of recent research, integrate the strengths of deterministic and intelligent methods in an attempt to overcome the limitations that each has on its own.

Finally, the choice of an appropriate method depends strongly on the type of mission

(static or dynamic), on hardware limitations (processing and communication), and on system priorities (time versus energy consumption). For real-time applications and dynamic environments, hybrid and AI-based methods (especially PSO and RRT*) are considered the superior options.

CHAPTER 4

Challenges and Applications of Path Planning in Swarm Robots

This chapter examines the complexities of deploying swarm robots, particularly with respect to their navigation and coordination, and then reviews the main domains in which swarms are being applied.

4.1 Challenges in Dynamic and Complex Environments

With complex obstacles, large search spaces, and unpredictable changes, the difficulty of path planning increases dramatically. In multi-robot systems, simultaneous operation creates a risk of collisions between the robots and with obstacles, which can lead to equipment damage, delays, or mission failure; collision avoidance therefore becomes a critical challenge [14], [101].

4.1.1 Obstacle Avoidance and Collision Resolution

In the swarm robotics community, obstacle avoidance is regarded as a basic and important task. It refers to the ability of robots to move around obstacles without colliding with them, mimicking the maneuverability observed in biological swarms [37].

4.1.1.1 Collision Avoidance Strategies in Dynamic Environments

In a dynamic environment, the ability of the planning algorithm to adjust the path in real time to accommodate moving obstacles, changes in the layout of the environment, or relocated targets is essential. The challenge intensifies when the moving obstacles have unknown headings, which makes accurate prediction of potential blocking points difficult [23].

The strategies used for collision avoidance in dynamic multi-robot systems typically emphasize real-time adaptability and computational efficiency:

- **Decentralization:** The strategies are generally distributed and based on local information and communication constraints, which contributes to robustness and adaptability [101].
- **Real-time replanning:** The rapidly-exploring random tree (RRT) algorithm and its variants have been adapted to operate in real time in dynamic environments. One strategy grafts new waypoints onto the existing path based on local obstacle avoidance; by avoiding the recomputation of the entire original path when an obstacle is detected, it greatly reduces the computation time and reaches the corrected path with a relatively short deviation from the initial one [23].
- **Learning-based methods:** Deep reinforcement learning (DRL) methods are increasingly preferred over classical methods because they handle collision avoidance in high-dimensional spaces without requiring the robots' kinematics or a global map [19], [94].

4.1.1.2 Potential Force Methods for Collision Avoidance

The artificial potential field, introduced by Khatib in 1986, is the classical basis of real-time collision avoidance: the goal attracts, the obstacles repel, and the robot moves under the resulting force [88].

- **Avoidance mechanism:** Repulsion from an obstacle has been likened to the mechanism of an atomic nucleus: the obstacle generates a virtual repulsive force, and the robots maneuver around it like electrons [37].
- **Priority and guarantee:** The collision avoidance mechanism must take precedence over all other actions [30]. With properly defined potential functions, a mathematical guarantee can be given that no collision will occur, provided that the repulsive force becomes unbounded as the distance tends to zero [88].
- **Swarm coordination:** Potential functions are used to guide swarms and to modify the desired positions of the robots so that they circumvent obstacles; shape controllers combine several potential forces to reach a formation while avoiding collisions [37].

4.1.1.3 Obstacle Enlargement and Robot Shrinking

A simplifying deterministic technique in path planning is to adjust the effective size of the obstacles or of the robot in order to simplify collision checking [14].

- **Principle:** This mechanism achieves collision avoidance by effectively enlarging the obstacles and shrinking the moving body to a point mass.
- **Application:** In constrained optimization formulations (such as global path planning in a static environment), the radii of convex polygonal and circular obstacles are increased in proportion to the physical dimensions of the robot. The robot is thus modeled as a particle that remains collision-free as long as it stays outside the boundary of the enlarged obstacle [37].

4.1.1.4 Optimal Reciprocal Collision Avoidance (ORCA)

ORCA is a powerful and widely used variant of the reciprocal velocity obstacle (RVO) methods for collision-free navigation [102].

- **Mechanism:** Assuming that the velocities of the neighbors are known and exchanged, ORCA selects a collision-free velocity for each agent in real time by solving a small linear program over half-plane constraints.
- **Efficiency:** For an agent with k neighbors, ORCA takes $O(k)$ time [103].
- **Comparison with Voronoi-based methods:** ORCA is often compared with the buffered Voronoi cell (BVC) method, which has the same complexity and similar empirical performance. The key difference lies in the exchange of information: ORCA relies on communication to share positions and velocities, whereas BVC needs only the sensed relative positions to guarantee avoidance [103].

4.1.2 Unknown and Dynamic Environments

The transition from controlled laboratory settings to real-world applications requires handling environments whose information is incomplete or that change continuously. Under these

conditions, swarm path planning must address real-time search, robust coordination, and dynamic target seeking.

4.1.2.1 Search-and-Rescue Missions in Hazardous Conditions

Search-and-rescue (SaR) operations are a key target for swarm robotics, because these scenarios are inherently complex and hostile and challenge the application of traditional multi-robot systems [37].

- **Characteristics of SaR environments:** Large extents, from hundreds of square meters to hectares; strong dynamics caused by the agents themselves and by external events such as collapses, fires, floods, and earthquakes; and harsh conditions, including abrasive dust, water, chemicals, and numerous obstacles that damage the hardware and disrupt communication [37].
- **Advantages of swarms:** Robustness to the failure of units, scalability, and reliance on self-organization improve the emergency response through damage assessment, resource management, and movement through complex environments to find and assist survivors. Examples include aquatic drones searching flooded areas and nano-robots proposed for cleaning up hazardous materials [24].

4.1.2.2 Adapting to Moving Targets and Changing Conditions

An important challenge arises when the targets, the obstacles, or the terrain are not fixed but change dynamically and unpredictably [14], [37].

- **Handling dynamic changes:** Swarm systems must react and adapt in real time to changes in the terrain, in environmental conditions (such as weather and currents), or to unexpected obstacles, for example shifting debris after a disaster [24].
- **Moving target tracking (MTT):** In an environment full of obstacles, the swarm must simultaneously move as a flock, avoid the obstacles, and maintain enough collective coverage that a randomly moving target never leaves its sight; this problem is relevant to escorting, monitoring, and group transport [37].
- **Difficulty of prediction:** In environments with moving obstacles whose headings are unknown, determining the potential blocking points is difficult [23].
- **Adaptive algorithms:** Continuous motion and adaptation call for algorithms with predictive capabilities, such as bio-inspired optimizers that model warning behavior for cooperative planning in dynamic environments [14].

4.1.2.3 Real-Time Path Planning for Multi-Robot Systems

In dynamic, unknown, or partially known environments, computing an optimal global path is often prohibitively expensive, so the focus shifts to efficient real-time local strategies for planning and replanning:

- **Dynamic grid search (D* and D* Lite):** D* extends A* to changing maps: by updating the weights of the edges affected by newly observed obstacles, it repairs only the necessary parts of the path instead of replanning completely [104], [105].
- **Time rolling windows:** Combined with local optimizers such as ABC, PSO, or the fireworks algorithm, a rolling time window in which each robot perceives and predicts

its surroundings replaces costly global planning in dynamic and unknown environments [37].

- **RRT-based real-time replanning:** The RRT family is well suited to dynamic environments because it accounts for the dynamic constraints of the robot; grafting adds new waypoints to the existing path, which reduces the computational load of a complete recomputation [23].
- **Artificial potential field:** The APF is valuable for rapid path adjustment because of its fast reaction to nearby obstacles, but it is of limited use in highly complex environments because of the risk of getting trapped in local minima (Figure 3.8) [18].

4.1.3 Adaptation and Real-Time Replanning

The growing demand for deploying multi-robot systems in complex real-world scenarios has made it necessary to move from offline, static planning to strategies for real-time adaptation and rapid replanning.

4.1.3.1 Dynamic Multi-Robot Systems and Environmental Changes

Path planning in dynamic environments is multifaceted: the environment changes because of external factors, moving targets, or the actions of the robots themselves [37].

- **Need for real-time reaction:** Algorithms must adapt quickly to changes; traditional deterministic methods often fail to adapt rapidly to unexpected obstacles and are computationally expensive [15].
- **Environmental instability:** In applications such as SaR after natural disasters, the environment can change rapidly (falling debris, weather conditions, unexpected events), and the algorithms must allow the behavior and the flight or motion patterns to be changed immediately [24].
- **Adaptability and flexibility:** The swarm must be flexible enough to cope with various tasks and situations and to adjust its paths as water or air currents change or as obstacles appear [2].
- **Complexity of distributed control:** Controlling groups of simple, reactive robots in a distributed structure is difficult, because the desired macroscopic behaviors emerge only from local interactions, and path planning must respond to continuously changing scenarios without relying on a limited set of predefined conditions [3].

This demand for adaptability has pushed research toward learning-based methods, which let robots improve their strategies from experience [20].

4.1.3.2 RRT-Based Algorithms for Replanning

The RRT family and its variants are suitable for dynamic environments because of their ability to generate feasible paths and to incorporate the dynamic constraints of the robot directly [23].

- **RRT-Connect and optimality:** RRT and its enhanced versions, such as RRT-Connect and RRT*, have been explored in dynamic environments; although RRT* provides asymptotic optimality, it can be computationally expensive [13].

- **Grafting for real-time efficiency:** Grafting adds new points based on local avoidance and nonholonomic constraints, generating a new point when a potential collision is detected in the next sampling step. Instead of recomputing the whole tree, only a short detour results [23].

Dynamic variants include [13]:

- **DRRT:** Provides fast, feasible, but sub-optimal paths by pruning and regrowing the affected parts of the tree.
- **RRT*FND:** Achieves asymptotic optimality through repeated updates in a dynamic environment, albeit at a high computational cost.
- **RT-RRT*:** Retains the full tree structure without pruning, which enables fast queries for multiple goals and rapid rewiring of the path in response to dynamic obstacles and changing goals.

4.2 Scalability and Robustness

Scalability and robustness are common and essential properties of swarm systems that enable them to operate effectively in complex real-world scenarios. Robustness refers to the ability of the system to function normally despite disturbances or individual failures, whereas scalability refers to the ability to maintain effective operation across different population sizes [1], [3].

4.2.1 Scalability in Swarm Systems

Scalability is both a critical challenge and a defining characteristic of swarm robotics: the system must work with arbitrary population sizes, from a few units to potentially thousands. Since the control method is usually identical for every member of the swarm, the system is designed to be highly scalable [1], [2].

4.2.1.1 Maintaining Performance as the Number of Robots Increases

Scalability is the capacity of the system to adjust the number of robots flexibly without a noticeable change in its overall functionality or an excessive drop in efficiency or reliability [1], [2].

- **Design principle:** Scalability stems from decentralization and from predominantly local interactions. Methods that rely on global broadcasting or on monitoring large parts of the swarm are avoided in order to preserve scalability, and local sensing and communication facilitate this property [3].
- **Ideal versus actual performance:** In the ideal case, a super-scalable system performs almost independently of the number of robots; in a merely scalable system, the loss of efficiency as units are added is small; in an unscalable one, population growth causes severe degradation or failure [2].
- **Cognitive complexity:** For independently operated robots, the operator's workload grows as $O(n)$, whereas with proper distributed coordination, swarm control can bring it down to $O(1)$ [45], [106].

- **Effect on tasks:** Adding or removing members should not jeopardize the completion of the mission, and implicit reallocation of tasks should occur without external intervention [2].
- **Challenges:** As the number of robots increases, maintaining productivity becomes harder and requires mechanisms such as control hierarchies and clustering to manage the communication and processing load and to prevent bottlenecks [24].

4.2.1.2 Simulation of Large-Scale Swarms

Because swarm algorithms are meant to scale to populations that are hard to build physically, simulating hundreds or thousands of robots is a vital part of research, and modeling and simulation are the necessary tools for this purpose [1], [10]. Notable simulators and platforms include:

- **ARGoS:** A modular simulator that can run several physics engines in parallel; it can simulate 10,000 e-pucks in 60% of the time the real experiment would take [1], [107].
- **Player/Stage/Gazebo:** An open-source framework with multi-robot capabilities; for 2-D swarm behaviors, its run time scales approximately linearly up to at least 100,000 simple robots [37].
- **Kilobot:** A low-cost, small physical platform built to test collective algorithms with hundreds to thousands of robots; a thousand Kilobots have self-assembled into programmed shapes [108].

4.2.1.3 Exponential Growth of States in Modular Robotics

Although scalability usually refers to functioning independently of the size of the population, in modular robotic systems the complexity of a configuration space with many degrees of freedom poses a distinct challenge [109].

- **Modular systems:** Self-reconfigurable systems such as M-TRAN, ATRON, and PolyBot are designed to build and form complex structures.
- **State explosion:** The number of possible configurations grows exponentially with the number of modules, which makes centralized planning and control difficult and reinforces the need for robust local rules.

If each of n modules can be in one of k states, the number of combinations is

$$S = k^n, \tag{4.1}$$

before connectivity constraints are taken into account. Chain-type modular robots are particularly versatile, but managing this explosion remains a key computational obstacle [109].

4.2.2 Robustness and Fault Tolerance

Robustness and fault tolerance are fundamental properties of swarm robotic systems; they mean maintaining operation and a desired level of performance in the face of internal failures or external disturbances [1], [3].

4.2.2.1 Robustness to Failures of Individual Robots

Robustness to individual failures stems mainly from redundancy and from the absence of a centralized point of control [1], [2].

- **Massive redundancy:** The large number of robots provides critical redundancy for safety and fault tolerance, and the loss of one robot can be compensated for by the others.
- **Distributed control:** The absence of a single point of failure means that the failure of one node or part does not disable the whole system.
- **Performance preservation:** The design treats individuals as dispensable, so a swarm can lose a considerable share of its robots before the mission has to be abandoned, albeit with reduced capability [37].
- **Fault detection:** The natural redundancy of the swarm can be used to detect faulty robots; for example, robots that flash in synchrony, like fireflies, can flag a peer that falls out of step as faulty [3].

4.2.2.2 Fault Tolerance and Redundancy in Communication

Ensuring the robustness of communication in dynamic and crowded environments is challenging and requires special mechanisms to cope with failures, interference, and delays [24].

- **Redundant communication paths:** Establishing alternative paths helps maintain the robustness of the network if one path is broken.
- **Distributed nodes and reconfiguration:** Distributing communication tasks among several robots and reconfiguring the network dynamically adapt it to changing conditions and reduce the impact of faults.
- **Information redundancy:** Replicating critical information across several robots preserves its availability in the face of communication problems.
- **MANETs and multi-connectivity:** In missions such as SaR, the robots must maintain a mobile ad hoc network (MANET) without pre-existing infrastructure, using attractive forces and redundancy to stay connected. The notion of k-connectivity generalizes bi-connectivity: the network stays connected as long as fewer than k nodes fail, that is, when its vertex connectivity satisfies $\kappa(G) \geq k$ [37].

4.3 Task Allocation and Coordination

Multi-robot task allocation (MRTA) is the problem of assigning tasks to individual robots so as to maximize the expected performance of the overall system, and it is usually treated as an instance of the optimal assignment problem. In swarm robotics, this coordination is vital for distributing the work and ensuring that collective goals are achieved efficiently under decentralized control [3], [33].

4.3.1 Dynamic Task Allocation

Dynamic task allocation refers to strategies in which the matching between tasks and robots is computed repeatedly over time, so that the robot team adapts to changing conditions,

tasks, or environments; online allocation can react to such changes, whereas simple offline methods cannot [33].

4.3.1.1 Efficient Achievement of Collective Goals

Achieving collective goals requires a distribution of tasks that takes into account the capabilities of the robots, their workload, and the environmental constraints, so as to maximize the performance of the whole swarm and minimize unnecessary redundancy [3].

- **Task partitioning:** Decomposing tasks greatly improves efficiency. In sequential tasks, robots can measure how long they wait for another subtask to be completed and use this local cue to decide whether to partition the work, so that interdependent subtasks run concurrently and interference is reduced [3].
- **Decentralized decision-making:** For scalability and robustness, each robot selects and prioritizes tasks based on its own state, its sensor data, and the mission goals [24].
- **Optimization objectives:** Criteria such as minimizing the completion time, maximizing resource utilization, or balancing the load among robots are used; maximizing collective utility when group dynamics affect performance is in general NP-hard [33]. Consensus-based allocation has been used to make the assignment both fast and energy-efficient [110].

4.3.1.2 Heuristic Algorithms for Distributed Task Selection

The distributed task selection problem in a swarm is NP-complete, so polynomial-time heuristics are used to obtain good-enough solutions [30].

- **Model and objective:** The mechanism tries to assign the desired number of swarm units to each task while reducing inter-task delays and communication overhead.
- **Performance of the heuristics:** Heuristics in which each unit considers both the effect of the other units on the tasks and its own relative position give better task processing and a better distribution of units over the tasks.
- **Communication constraints:** These approaches work without a central location or shared memory, using only distributed communication.

Auction- and market-based methods are the main alternatives to such heuristics [33].

4.3.1.3 Task Scheduling in Layered Architectures

In hierarchical or layered architectures, a scheduling layer manages the overall flow of the mission: at each instant, it selects the subset of tasks that should run concurrently, elects the robot that coordinates them, and decides how many tasks to request from each deployment point. A global planner may propose the allocation, which the scheduling layer then distributes to the swarm; the point is to approximate a good strategy for the robots' navigation controllers at a small computational cost while respecting the timing and ordering constraints of the tasks [33], [34].

4.3.2 Cooperative Control and Manipulation

Cooperative control is the problem of steering a multi-agent system toward a common goal (exploration, search, rescue, etc.); integrating multiple robots increases capacity, flexibility,

and efficiency [33].

4.3.2.1 Multi-Robot Teamwork Strategies

For cohesive and efficient operation, multi-robot systems require coordination strategies that ensure inter-robot communication, effective task allocation, and conflict resolution [33].

- **Decentralized coordination:** For scalability and robustness, coordination must be decentralized and based on local interactions and simple rules, so that global behaviors emerge from local interactions [3].
- **Implicit versus explicit:** Cooperation can emerge implicitly from the autonomous decisions of individuals or be supported by explicit communication (subject to bandwidth constraints) to improve information exchange and motion synchronization.
- **Behavioral approaches:** Behavior-based formation control and distributed control manage interactions and resolve conflicts with local information; a controller based on several potential forces can steer the robots toward the desired shape and prevent collisions [37].
- **Hierarchical architectures:** Hybrid-HASP (Section 2.2) combines competitive and cooperative mechanisms to select and combine elementary behaviors, and it supports cooperation through implicit communication with simple attractive and repulsive signals [37].

4.3.2.2 Cooperative Grasping and Object Transport

Cooperative transport and manipulation (e.g., box pushing) is a common benchmark and usually involves objects that a single robot cannot handle [3], [8].

- **Physical requirements:** A physical connection (self-assembly) is necessary to cross large gaps or to perform complex transport; cooperative grasping requires communication and docking before execution.
- **The Swarm-bot project:** This project showed that autonomous agents relying solely on local information can cooperatively transport heavy objects that a single robot cannot move; the aggregation of s-bots into a formation plays a key role [46], [111].
- **Coordinated motion:** In cooperative transport, the robots must share information about the object and the destination, agree on a common direction, and then move together; Hybrid-HASP has been used to coordinate box pushing [37].

4.3.2.3 Consensus, Tracking, and Containment

Path planning for cooperative swarms must address the fundamental objectives of consensus, maintenance of relative positions or formation (tracking), and containment [49].

- **Consensus control:** The group must reach agreement on the variables of interest; consensus algorithms, close relatives of distributed linear algorithms, typically move each agent toward the average of the values it receives [48].
- **Tracking and formation control:** Formation is a key application of cooperative control whose aim is to achieve a fixed geometric pattern; leader–follower and flocking approaches are used, the latter with collision avoidance, velocity matching, and flock centering [11], [49]. A simulation of leader–follower tracking is shown in Figure 4.1.

- **Containment and region control:** In region control, the group is kept within a predefined region using constraint functions, and in multilevel designs, the robots converge to the region and form the shape level by level [37].

With several leaders, containment requires every follower to stay in the convex hull spanned by the leaders,

$$x_i(t) \in \operatorname{conv}\{x_j(t) : j \in \mathcal{L}\}, \tag{4.2}$$

where $\mathcal{L}$ is the set of leaders [49].

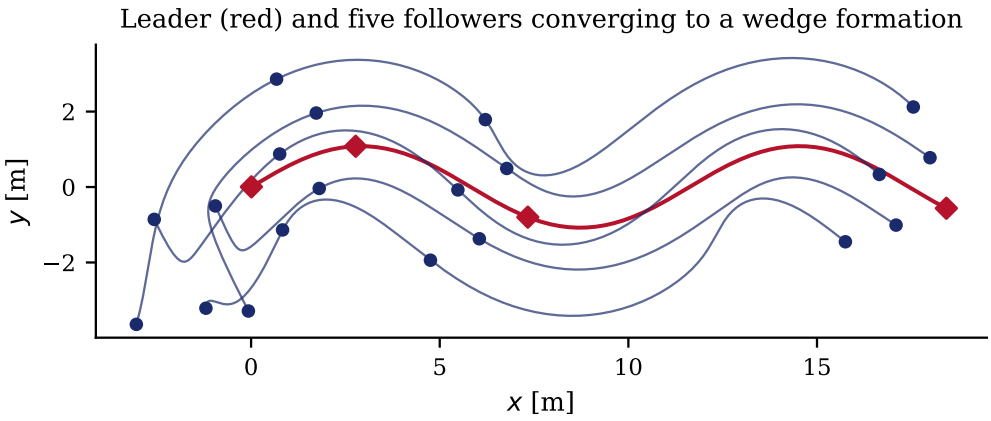


Figure 4.1. Leader–follower formation tracking: five followers start from random positions and converge, through local consensus on their formation errors, to a wedge around the leader (markers every few seconds).

4.4 Real-World Applications of Swarm Robotics

The ultimate goal of swarm robotics research is to deploy these systems in real-world scenarios, especially where the tasks are too hazardous, complex, or large-scale for individual robots or human teams. Although swarm robotics has long been confined to academia, the field is moving toward practical applications [5], [112]. Examples are compiled in Table 4.1 and discussed below.

4.4.1 Search and Rescue

4.4.1.1 Exploring Hazardous Environments

Disaster sites are hostile, dynamic, and large, with dust, water, chemicals, and debris, and these conditions cannot be handled by simply adapting traditional planning and control [37]. Swarms survive such conditions through redundancy, and they can map collapsed structures and identify hazards, locate trapped survivors by body heat, heartbeat, or sound, and track chemical or radiation leaks [24]. Proposed and demonstrated examples include insect-sized flying robots (RoboBees) for searching collapsed buildings, aquatic drones (AquaDrones) and amphibious robots for floods, and swarms for demining [24].

4.4.1.2 Initial Deployment and Communication in Faulty Environments

Two open problems in real SaR missions are the initial deployment of the robots and the integrity of their network [37]. The initial placement strongly affects the efficiency of the

Table 4.1. Examples of real-world applications of swarm robotics.

Application domain	Specific tasks and missions	Example platforms or projects	Key advantages and methods
Agriculture	Precision weeding, crop monitoring	LettuceBot, senseFly [24]	Less herbicide through machine vision; continuous monitoring
Search and rescue	Locating survivors in rubble, flood response	RoboBees, AquaDrones [24]	Rapid area coverage; operation without GPS or infrastructure
Construction	Bricklaying, site inspection	Skycatch [24]	Parallel work and faster construction
Environmental monitoring	Water quality, marine pollution	microUSV, CoCoRo [1], [5]	Distributed sensing and collective mapping
Warehousing and logistics	Goods handling in warehouses	Kiva [5]	Decentralized path coordination in crowded spaces
Space exploration	Resource prospecting, in-space assembly	Swarmies, Marsbees [5]	Redundancy and fault tolerance far from help

search; options include random deployment, contextual deployment that uses map or network-quality information, and marsupial deployment, in which a larger robot carries and releases smaller ones. Communication is hard because obstacles are everywhere, the topology of the MANET keeps changing, and low batteries make nodes fail. Simple range-based rules are unreliable, so approaches refresh connectivity tables continuously and limit the robots' motion to keep multi-hop routes of sufficient quality. Virtual attractive forces based on signal strength keep a robot close to its neighbors while repulsive forces prevent collisions, and redundancy lets the remaining robots fill the gap left by one that loses contact [37].

4.4.2 Environmental Monitoring and Mapping

4.4.2.1 Mapping Scalar Fields and Marine Environments

A common mission is to map an unknown scalar field, such as temperature, salinity, or pollutant concentration, over a large area, which suits mobile robot networks. In one cooperative scheme, each robot runs two distributed consensus filters: a weighted-average consensus filter estimates the field value, an average consensus filter estimates the confidence of that estimate, and the map is refined with a weighted-average protocol as the robots move [37]. At sea, the CoCoRo project used 41 heterogeneous underwater vehicles, communicating through electric fields and sonar, for environmental monitoring, while the Platypus system uses swarms of surface vessels to map water quality and stratify salinity and oxygen levels [5].

4.4.2.2 Chemical Plume Tracing and Source Localization

Chemical plume tracing (CPT) steers robots, instead of people, toward the source of toxic gas leaks, fires, or explosives. It is hard in natural environments because the source strength and the advection distance are unknown and the plume is intermittent and meandering under turbulent, time-varying flow. CPT strategies are often taken from insects, notably moths,

using anemotaxis, chemotaxis, and their combination. In swarm CPT under formation control, a leader performs the moth-inspired maneuvers and the others keep the formation; robots that can switch roles outperform a single robot. Since physical experiments are hard to repeat, simulation platforms based on computational fluid dynamics are essential for testing these algorithms under identical conditions [37].

4.4.2.3 Area Coverage for Environmental Assessment

Coverage requires a group of robots to visit every position of an area, usually discretized into a grid; in dynamic coverage, robots with limited-range sensors keep moving and sampling until a required coverage level is reached [113]. ACO-style virtual pheromones mark explored areas so that others avoid redundant visits, and PSO-style updates spread the robots over large areas [6]. For photovoltaic parks, a multi-objective ACO planner adapts the flight paths to wind and moving obstacles and improves both travel time and coverage (Section 2.4) [57].

4.4.3 Agriculture

Swarm robotics addresses labor shortages and the need for sustainable food production [12], [24].

- **Harvesting and plant care:** Swarm systems for autonomous harvesting use computer vision and machine learning to pick ripe crops and a decentralized architecture for flexibility and robustness, which reduces labor costs and post-harvest losses [24].
- **Weeds and pests:** LettuceBot distinguishes lettuce from weeds with computer vision and sprays herbicide only on the weeds; robots that target pests individually reduce the use of chemicals [24].
- **Precision agriculture:** Robots and drones guided by swarm intelligence monitor soil and crop health in real time, seed at accurate intervals and depths, and adapt irrigation and fertilization to soil moisture, while the swarm scales to large farms [12], [24].

4.4.4 Space Exploration and Construction

Space missions are risky, remote, and subject to long communication delays, which favors autonomous, robust, and scalable swarms [5].

- **Exploration and resource discovery:** Swarms of small explorers can map planetary surfaces and analyze terrain in place of a single large rover, and they can prospect for water, ice, and minerals for in-situ resource utilization; NASA's Swarmies were built for this purpose, and the Marsbees concept proposes bumblebee-sized flapping-wing flyers for Mars [5].
- **Modular robots in space:** Modular reconfigurable robots are versatile, robust, and compact: one system can replace many specialized tools, and spare modules allow self-repair, which saves mass and volume. PolyBot-type modules can serve for manipulation and surface mobility [109], [114].
- **Construction and assembly:** Self-assembly, demonstrated by the Swarm-bot project, lets robots overcome individual limitations and is a model for on-orbit assembly; the heterogeneous Swarmanoid combined flying, climbing, and ground robots to work in three dimensions [10], [111].

4.4.5 Medical Applications

Micro- and nanorobots may bring swarm principles into medicine [24].

- **Minimally invasive surgery:** Groups of very small, maneuverable robots could reach hard-to-access regions through small incisions and cooperate in tissue manipulation or suturing. Untethered tools such as microgrippers that respond to temperature, pH, or enzymes have performed in-vitro biopsies, and magnetically steered microdrillers have cut tissue ex vivo.
- **Targeted drug delivery:** Miniature robots could navigate to a target and release drugs locally, raising efficacy and reducing side effects.
- **Monitoring and diagnosis:** Swarms could take diagnostic samples and track vital signs, but localization, actuation, and control in vivo remain open problems [24].

4.4.6 Object Transportation and Manipulation

Collective transport exploits many robots to move objects that are too heavy or complex for one [3]. Ants pull at the front of a load and push at its back, a pattern quite different from solitary transport [8]. Very large robotic systems with hundreds or thousands of robots, for example for transport and assembly, call for macroscopic models of the group [115]. The Swarm-bot experiments showed cooperative transport with local information only [46], [111], and cooperative box pushing has served as a benchmark for reactive and distributed architectures, including Hybrid-HASP [37].

4.4.7 Security and Defense

4.4.7.1 Key Predistribution for Secure Communication

Swarm robots often lack the memory, computation, and energy for public-key cryptography, so key predistribution schemes (KPS) are an alternative: a trusted authority installs key material on each robot before deployment, and pairs of robots that share keys can derive secret keys and authenticate each other as legitimate members. A KPS trades off memory, connectivity, and resilience: more keys per robot improve connectivity but expose more of the network when a robot is captured, and fewer shared keys slow the swarm down because robots can only talk to neighbors with a common key. Schemes are probabilistic (random key subsets) or deterministic (combinatorial designs) [37].

4.4.7.2 Reconnaissance, Surveillance, and Target Acquisition

Swarms of UAVs and ground vehicles are used for perimeter monitoring and tactical reconnaissance without risking people; DARPA's OFFSET program uses UAVs and UGVs for observation and hazard detection in cities, and a military swarm of 103 drones has been demonstrated without a single controlling leader [5]. Digital pheromones are a natural fit: unmanned vehicles climb attractive gradients centered on targets and avoid repulsive gradients centered on threats such as surface-to-air missiles [4].

4.4.8 Logistics

In warehouses, fleets such as Kiva move goods with decentralized path coordination in crowded spaces [5], and swarm-intelligence optimizers are widely used for routing and

allocation problems in logistics [12].

4.4.9 Patrolling and Surveillance

4.4.9.1 Monitoring Restricted Areas

Surveillance means searching an area persistently, so that every zone is revisited often enough for the information to stay current; patrolling is the special case in which the area reduces to points or lines, such as building corridors [56]. Swarms patrol industrial plants, public spaces, and borders [24].

4.4.9.2 Comparison with Search Tasks

Search looks for specific targets in an area; exploration has no prior map and aims to build one; coverage means either spreading agents to maximize their footprint or visiting every cell of a grid [56]. Search algorithms can be adapted to surveillance: a random walk revisits areas statistically, algorithms that track visited cells can use the time since a cell was last seen (its "age") instead of a visited flag, and lane-based search can number its lanes by visit order. For surveillance, the maximum age of the cells is a convenient performance measure [56].

4.4.10 Data Analysis and Graph Partitioning

Swarm intelligence has also produced data-analysis methods. Ants cluster corpses into cemeteries and sort their brood, moving items according to the local density of similar items. Lumer and Faieta turned this into an exploratory data-analysis algorithm in which similar objects end up close together, and Kuntz, Layzell, and Snyers adapted it to graph partitioning, where clusters of tightly connected vertices emerge from local actions; templates make the number and location of clusters controllable [8]. Clustering and sorting by robots became early benchmarks of swarm robotics for the same reason [8], [10].

CHAPTER 5

Summary, Conclusions, and Future Work

5.1 Summary of Path Planning Approaches

Path planning is a vital component of swarm operations that fundamentally affects mission success, energy consumption, and the efficiency of the whole system. Advancing these algorithms—often driven by biological inspiration and modern AI techniques—is essential for realizing the full potential of autonomous systems.

5.1.1 Overview of Deterministic, AI-Based, and Hybrid Approaches

According to their core methodology, the path planning techniques developed for swarm systems fall into three main categories: deterministic approaches, AI-based approaches, and hybrid approaches.

Deterministic approaches. These are classical algorithms that rely on established mathematical models and predefined rules to generate paths. Although these methods often provide globally optimal paths and a straightforward implementation, they face serious difficulties in dynamic and unpredictable environments and can be computationally expensive; hence, on their own, they are not commonly used for real-time applications.

AI-based approaches. These methods use machine learning and other intelligent technologies to adapt to dynamic and uncertain environments by learning from data or experience [15]. Their key advantage is the ability to handle complex scenarios, to make intelligent decisions in real time, and to adapt quickly to unexpected obstacles or changes in the environment. However, they generally require extensive training data and substantial computational resources.

Hybrid approaches. These strategies integrate elements of both deterministic and AI-based methods in order to exploit their complementary strengths and compensate for the limitations of each. The goal is to produce solutions that are more robust, adaptive, and efficient in complex and dynamic environments. Studies show that hybrid approaches make up roughly 25% of the reviewed path planning literature, which indicates an open research space for advancing the field [15].

5.1.2 Methodologies and Their Principles

The core principles of these three families of approaches reveal their distinct contributions to solving the path planning problem in swarm robotics.

Principles of deterministic methods. Deterministic methods usually transform the continuous motion planning problem into a discrete search problem.

- **Graph-based methods (e.g., A*):** These methods determine the optimal path on a discrete map using heuristic cost functions. A common formulation of A* uses the function $f(n) = g(n) + h(n)$, where $g(n)$ is the accumulated cost and $h(n)$ is the heuristic estimate of the cost to the goal.
- **Sampling-based methods (e.g., PRM, RRT, RRT*):** These methods explore complex, high-dimensional spaces through random sampling. They are usually computationally efficient and probabilistically complete, but they do not guarantee global optimality.
- **Artificial potential fields (APF):** These methods generate the path from geometric principles, using a virtual force that attracts the robot toward the goal and repels it from the obstacles. Conceptually, the total potential can be written as $U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q)$ and the guiding force as $F(q) = -\nabla U(q)$, so that the path is shaped by gradient descent.

Principles of AI-based methods. These methods rely on learning and on iterative optimization inspired by natural processes; in path planning, bio-inspired techniques have been the most widely used.

- **Particle swarm optimization (PSO):** A stochastic algorithm based on social behavior that balances the exploitation of the best local position with the exploration of the best global position. The standard formulation consists of the velocity and position updates

$$\begin{aligned} v_i^{t+1} &= \omega\, v_i^t + c_1 r_1 \left(pbest_i - x_i^t\right) + c_2 r_2 \left(gbest - x_i^t\right), \\ x_i^{t+1} &= x_i^t + v_i^{t+1}, \end{aligned} \tag{5.1}$$

  which make clear the roles of the inertia weight ω, the acceleration coefficients c_1 and c_2, and the random factors r_1 and r_2 in controlling the balance between exploration and exploitation.
- **Ant colony optimization (ACO):** Agents mark and follow desirable paths using an artificial pheromone, which reduces redundant motion and improves coordination. The common evaporation and reinforcement rule is $\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij} + \sum_k \Delta\tau_{ij}^k$ with $\Delta\tau_{ij}^k = Q/L_k$ on the traversed edges, which facilitates convergence to shorter paths.
- **Reinforcement learning (RL):** Agents learn an optimal control policy through trial-and-error interaction and by receiving rewards or penalties, which makes RL suitable for dynamic and uncertain environments.

Principles of hybrid methods. Hybrid methods integrate the efficiency of one algorithm with the specialized capability of another. For example, combining a deterministic algorithm with an AI technique can be used to tune the parameters adaptively or to guide the search process. A prominent example is the enhancement of a fuzzy neural network (FNN) path planner by integrating the A* algorithm to guarantee path optimality, which improves the rationality and feasibility of the path by removing limitations such as motion jumps.

5.2 Conclusion

The systematic review of path planning methodologies and real-world applications in this seminar confirms the vital position of swarm robotics as a key field in the advancement of

autonomous systems. Swarm systems are designed to overcome the inherent limitations of single-robot or traditional multi-robot configurations, and they draw on the principles of spontaneous self-organization in nature and on computational intelligence.

5.2.1 Advantages of Swarm Robotics

- **Fault tolerance:** Swarm systems are inherently robust owing to their extensive redundancy: the failure of one robot does not undermine the integrity of the mission. By distributing control and functionality across many agents and eliminating single points of failure, high robustness and reliability are achieved.
- **Scalability:** A central feature of swarm robotics is efficient scaling from a few robots to thousands while maintaining performance and efficiency. This scalability stems directly from local, simple, and distributed coordination mechanisms.
- **Productivity and parallelism:** Swarms have an advantage in complex, large-scale problems, performing tasks that are impractical for a single robot. Their algorithms are generally designed to be parallel, so that multiple goals can be handled simultaneously, yielding significant time savings in search, exploration, and construction.
- **Decentralization and simplicity:** Decentralized control and spontaneous self-organization produce macroscopic behaviors from simple local interactions. The simpler components of individual robots make development easier and cheaper than that of complex centralized units.

5.2.2 The Critical Role of Advanced Path Planning

- **The need for safe and reliable motion:** Path planning is vital for ensuring safe, reliable, and smooth motion, especially in large-scale or obstacle-rich environments. It directly affects mission success, resource utilization, and energy consumption.
- **Leveraging collective intelligence:** Advanced path planning based on swarm intelligence principles, such as ACO, PSO, and RL, is essential for realizing the potential of swarm systems. By extracting insight from experience and data, these methods provide intelligent adaptation in dynamic and uncertain environments—a capability that traditional deterministic methods alone lack.
- **Hybridization for robustness and efficiency:** Developing hybrid approaches is key: the optimality of deterministic methods is combined with the flexibility of AI-based methods [15]. A prominent example is the integration of a fuzzy neural network (FNN) with the A* algorithm, which guarantees path optimality and improves feasibility and controllability by removing the inherent limitations of the FNN.
- **Scalability through virtualization and optimization:** Advanced methods must cope with the severe computational complexity of path planning for large swarms. The virtual tube optimization method, by combining a centralized definition of the path with distributed model predictive control (MPC), drastically reduces the computational cost through simple interpolation of the optimal path and preserves scalability in obstacle-rich environments.

5.3 Future Research Directions

Future research must give swarms the means to handle increasingly complex tasks in uncertain, dynamic, real-world environments, mainly through better control, learning, and interaction mechanisms [24], [112].

5.3.1 Advanced Control and Learning Strategies

5.3.1.1 Adaptive Learning and Self-Organization

- **Learning from experience:** Swarm robots should learn from their own experience and adapt their behavior to changes in the environment and in the task, which requires learning methods tailored to swarms rather than borrowed from single robots [20], [24].
- **Robust decentralized coordination:** Coordination strategies must stay robust under model uncertainty and sensor noise, and self-organizing behaviors should let the swarm reconfigure quickly when the mission changes [20].
- **Scalable learning:** Multi-agent reinforcement learning has to cope with joint action spaces that grow exponentially with the number of robots; representations that are invariant to the number and order of neighbors are one answer [19].
- **Hybridization:** Combining swarm metaheuristics (GA, PSO, ACO) with one another and with learned models is a promising way to keep their strengths and remove their weaknesses [12].

5.3.1.2 Evolved Controllers and Gene Regulatory Networks

Evolutionary robotics automates controller design by evolving, for instance, the weights and topology of neural networks, and it remains a main route to adaptive swarm behaviors [20]. A more specialized direction uses gene regulatory networks (GRNs) as controllers: robots driven by evolved, hierarchical GRNs have aggregated into larger organisms whose configuration adapts to a changing environment, with mutation and duplication operators improving that adaptability [24]. Genetic algorithms and genetic programming can also search for control strategies that suit a particular mission [20].

5.3.2 Human–Swarm Interaction

Even autonomous swarms need human supervision when objectives are complex or change during the mission [45].

- **Why humans stay in the loop:** Operators can spot and mitigate shortcomings of the autonomy, bring in information the robots cannot sense, and change the mission intent [45].
- **Cognitive complexity:** The goal is $O(1)$ control, where the operator's effort does not grow with the number of robots; measuring it properly requires notions of complexity that also account for task difficulty and the level of automation [45], [106].
- **Levels of automation and neglect benevolence:** Swarm interfaces sit high on the scale of automation. Because swarm dynamics take time to unfold, an operator may need to wait before a command can succeed, a property called neglect benevolence (the converse

of neglect tolerance); predictive displays that show the effect of a command before it is given help with this [45].

- **Interfaces:** Haptic feedback about nearby obstacles improves operator performance, especially combined with visual feedback; how to visualize noisy or partially observed swarm states remains open [45]. Brain–machine interfaces have also been proposed for human–swarm collaboration [24].
- **Shared control:** Blending operator commands with the robots' autonomy, so that the system asks for help only when needed, is an effective way to reduce the operator's burden [45].

5.3.3 Hardware and Simulation

5.3.3.1 Miniaturization and Sensing

Smaller, cheaper, and more capable robots would make large swarms practical. Accurate, noise-robust low-cost sensors, richer sensor combinations (cameras, LiDAR, acoustic, and chemical sensors), and radios with time-of-flight ranging for relative localization are all needed, and batteries remain too large for the smallest platforms [2], [112].

5.3.3.2 Simulation Platforms and the Reality Gap

Simulation is indispensable because large physical experiments are slow and expensive. Kilogrid extends Kilobot experiments with a virtual environment, the Pi-puck extension adds a single-board computer to the e-puck, and ARGoS simulates thousands of robots faster than real time [1], [112]. Controllers designed in simulation must survive the transfer to real robots, which calls for a hierarchy of models of increasing fidelity and for measurements of real swarms to calibrate the simulators [10], [20].

5.3.3.3 Three-Dimensional and Denser Swarms

Aerial platforms remove the planar restriction of most earlier work and bring swarms closer to bird flocks and fish schools, but 3-D sensing, communication, and control are harder, distributed 3-D localization is still needed, and the formation flight of large UAV swarms is far from solved [2], [58].

5.3.4 Ethics and Societal Impact

As swarms leave the laboratory, their safety around people, the privacy of the data they collect, algorithmic bias, and the accountability for decisions taken by a decentralized system become pressing questions. Privacy-preserving techniques such as federated learning, clear rules on data retention, and transparent descriptions of what a swarm can do are part of a responsible deployment, together with international norms that prevent misuse [12], [24].

5.3.5 Dynamic Formation Generation and Route Planning

Generating formation patterns on the fly and planning routes in unknown environments are named as open directions in formation control [43], [44]. Without a prior map, planning must rely on on-board sensing, and swarm data could feed highly dynamic maps that narrow the gap between map-based and purely reactive planning [37]. Multilevel topologies (Section 2.2)

and virtual linkages, which reshape a formation by changing a few joint angles, are natural building blocks for formations that change with the environment [37], [43].

5.3.6 Sensing-Aware Planning

- **Information-driven UAV paths:** Choosing waypoints that minimize the CRLB turns path planning into a constrained nonlinear program, solved for example with interior penalty methods; the determinant of the Fisher information matrix plays the same role for received-signal-strength localization. In simulation, four UAVs located both static and moving emitters this way [52].
- **Coverage with complex detectors:** Coverage with extended-range detectors uses the generalized Voronoi diagram to handle narrow regions [84], and heterogeneous swarms covering non-convex areas under visibility constraints remain an active topic [85].

5.3.7 Multi-Objective Planning and Constraint Handling

Real missions combine conflicting objectives, such as time, energy, and safety, so planners should return Pareto sets rather than single answers. MOPSO (Section 3.1) generates a set of non-dominated solutions in one run, and constraint-handling schemes such as the penalty mechanism of CAMOPSO, which lets marginally infeasible solutions return to the feasible region, deserve further study on constrained problems; fuzzy decision-making can then pick a compromise from the front [55].

References

[1] P. G. F. Dias, M. C. Silva, G. P. Rocha Filho, P. A. Vargas, L. P. Cota, and G. Pessin, "Swarm robotics: A perspective on the latest reviewed concepts and applications," *Sensors*, vol. 21, no. 6, 2021, Art. no. 2062.

[2] Y. Tan and Z.-y. Zheng, "Research advance in swarm robotics," *Defence Technology*, vol. 9, no. 1, pp. 18–39, 2013.

[3] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, "Swarm robotics: A review from the swarm engineering perspective," *Swarm Intelligence*, vol. 7, no. 1, pp. 1–41, 2013.

[4] E. Şahin and W. M. Spears, Eds., *Swarm Robotics: SAB 2004 International Workshop* (Lecture Notes in Computer Science). Berlin, Germany: Springer, 2005, vol. 3342.

[5] M. M. Shahzad et al., "A review of swarm robotics in a nutshell," *Drones*, vol. 7, no. 4, 2023, Art. no. 269.

[6] L. V. Nguyen, "Swarm intelligence-based multi-robotics: A comprehensive review," *AppliedMath*, vol. 4, pp. 1192–1210, 2024.

[7] H. Hamann, "Space-time continuous models of swarm robotic systems: Supporting global-to-local programming," Ph.D. dissertation, Universität Fridericiana zu Karlsruhe (TH), Karlsruhe, Germany, 2008.

[8] E. Bonabeau, M. Dorigo, and G. Theraulaz, *Swarm Intelligence: From Natural to Artificial Systems*. New York, NY, USA: Oxford University Press, 1999.

[9] S. Garnier, J. Gautrais, and G. Theraulaz, "The biological principles of swarm intelligence," *Swarm Intelligence*, vol. 1, no. 1, pp. 3–31, 2007.

[10] H. Hamann, *Swarm Robotics: A Formal Approach*. Cham, Switzerland: Springer, 2018.

[11] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," *ACM SIGGRAPH Computer Graphics*, vol. 21, no. 4, pp. 25–34, 1987.

[12] U. Seebacher and C. Legat, Eds., *Collective Intelligence: The Rise of Swarm Systems and their Impact on Society*. Boca Raton, FL, USA: CRC Press, 2025.

[13] W. Chen et al., "A survey of autonomous robots and multi-robot navigation: Perception, planning and collaboration," *Biomimetic Intelligence and Robotics*, vol. 5, 2025, Art. no. 100203.

[14] S. Lin, A. Liu, J. Wang, and X. Kong, "A review of path-planning approaches for multiple mobile robots," *Machines*, vol. 10, no. 9, 2022, Art. no. 773.

[15] N. AbuJabal, T. Rabie, M. Baziyad, I. Kamel, and K. Almazrouei, "Path planning techniques for real-time multi-robot systems: A systematic review," *Electronics*, vol. 13, no. 12, 2024, Art. no. 2239.

[16] P. Mao, R. Fu, and Q. Quan, "Optimal virtual tube planning and control for swarm robotics," *arXiv preprint arXiv:2304.11407*, 2023.

[17] P. Mao, S. Lv, and Q. Quan, "Tube RRT*: Efficient homotopic path planning for swarm robotics passing-through large-scale obstacle environments," *arXiv preprint arXiv:2404.09200*, 2024.

[18] D. Zhang, G. Zhu, and Q. Zhang, "Multi-robot motion planning: A learning-based artificial potential field solution," *arXiv preprint arXiv:2306.07647*, 2023.

[19] M. Hüttenrauch, A. Šošić, and G. Neumann, "Deep reinforcement learning for swarm systems," *Journal of Machine Learning Research*, vol. 20, pp. 1–31, 2019.

[20] J. Kuckling, "Recent trends in robot learning and evolution for swarm robotics," *Frontiers in Robotics and AI*, vol. 10, 2023, Art. no. 1134841.

[21] R. Azzam, I. Boiko, and Y. Zweiri, "Swarm cooperative navigation using centralized training and decentralized execution," *Drones*, vol. 7, 2023, Art. no. 193.

[22] J. Li and Y. Tan, "A two-stage imitation learning framework for the multi-target search problem in swarm robotics," *Neurocomputing*, 2019.

[23] Y. Li, "An RRT-based path planning strategy in a dynamic environment," in *2021 7th International Conference on Automation, Robotics and Applications (ICARA)*, Prague, Czech Republic, 2021, pp. 1–5.

[24] P. Bhowmick, S. Das, and F. Arvin, Eds., *Bio-Inspired Swarm Robotics and Control: Algorithms, Mechanisms, and Strategies*. Hershey, PA, USA: IGI Global, 2024.

[25] G. Beni and J. Wang, "Swarm intelligence in cellular robotic systems," in *Robots and Biological Systems: Towards a New Bionics?* Ser. NATO ASI Series, vol. 102, Berlin, Germany: Springer, 1993, pp. 703–712.

[26] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 – International Conference on Neural Networks*, 1995, pp. 1942–1948.

[27] J. Kennedy, R. C. Eberhart, and Y. Shi, *Swarm Intelligence*. San Francisco, CA, USA: Morgan Kaufmann, 2001.

[28] P.-P. Grassé, "La reconstruction du nid et les coordinations interindividuelles chez *Bellicositermes natalensis* et *Cubitermes* sp.," *Insectes Sociaux*, vol. 6, pp. 41–80, 1959.

[29] S. Nouyan, A. Campo, and M. Dorigo, "Path formation in a robot swarm: Self-organized strategies to find your way home," *Swarm Intelligence*, vol. 2, no. 1, pp. 1–23, 2008.

[30] E. Şahin, W. M. Spears, and A. F. T. Winfield, Eds., *Swarm Robotics: Second International Workshop, SAB 2006* (Lecture Notes in Computer Science). Berlin, Germany: Springer, 2007, vol. 4433.

[31] S. Gudeta, A. Karimoddini, M. Davoodi, and I. Raptis, "Leaderless swarm formation control: From global specifications to local control laws," in *2021 American Control Conference (ACC)*, New Orleans, LA, USA, 2021.

[32] L. Pan, Q. Lu, K. Yin, and B. Zhang, "Signal source localization of multiple robots using an event-triggered communication scheme," *Applied Sciences*, vol. 8, no. 6, 2018, Art. no. 977.

[33] Z. Yan, N. Jouandeau, and A. A. Cherif, "A survey and analysis of multi-robot coordination," *International Journal of Advanced Robotic Systems*, vol. 10, 2013.

[34] Z. Liu, X. Wang, J. Li, Y. Cong, and S. Zhao, "A distributed and modularised coordination framework for mission oriented fixed-wing UAV swarms," in *2019 Chinese Control and Decision Conference (CCDC)*, 2019.

[35] V. Rezaei and E. Khanmirza, "A novel optimal distributed strategy for time-varying formation tracking control in large-scale robot swarms," *Scientific Reports*, 2025.

[36] W. Tan, H. Wei, and B. Yang, "SambotII: A new self-assembly modular robot platform based on Sambot," *Applied Sciences*, vol. 8, no. 10, 2018, Art. no. 1719.

[37] Y. Tan, Ed., *Handbook of Research on Design, Control, and Modeling of Swarm Robotics*. Hershey, PA, USA: IGI Global, 2016.

[38] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.

[39] Y. K. Lopes, S. M. Trenkwalder, A. B. Leal, T. J. Dodd, and R. Groß, "Supervisory control theory applied to swarm robotics," *Swarm Intelligence*, vol. 10, no. 1, pp. 65–97, 2016.

[40] Y. C. Tan and B. E. Bishop, "Evaluation of robot swarm control methods for underwater mine countermeasures," in *Proceedings of the Thirty-Sixth Southeastern Symposium on System Theory*, 2004.

[41] H. Ge, G. Chen, and G. Xu, "Multi-AUV cooperative target hunting based on improved potential field in a surface-water environment," *Applied Sciences*, vol. 8, no. 6, 2018, Art. no. 973.

[42] G. C. Maffettone, L. Liguori, E. Palermo, M. di Bernardo, and M. Porfiri, "Hybrid platform for swarm robotics: Experiments and high-dimensional continuification control," *arXiv preprint arXiv:2310.01573*, 2023.

[43] Y. Liu, J. Gao, C. Liu, F. Zhao, and J. Zhao, "Reconfigurable formation control of multi-agents using virtual linkage approach," *Applied Sciences*, vol. 8, no. 7, 2018, Art. no. 1109.

[44] Y. Liu, J. Gao, X. Shi, and C. Jiang, "Decentralization of virtual linkage in formation control of multi-agents via consensus strategies," *Applied Sciences*, vol. 8, no. 11, 2018, Art. no. 2020.

[45] A. Kolling, P. Walker, N. Chakraborty, K. Sycara, and M. Lewis, "Human interaction with robot swarms: A survey," *IEEE Transactions on Human-Machine Systems*, vol. 46, no. 1, pp. 9–26, 2016.

[46] F. Mondada, A. Guignard, M. Bonani, D. Bär, M. Lauria, and D. Floreano, "SWARM-BOT: From concept to implementation," in *Proceedings of the 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Las Vegas, NV, USA, 2003.

[47] M. Mesbahi and M. Egerstedt, *Graph Theoretic Methods in Multiagent Networks*. Princeton, NJ, USA: Princeton University Press, 2010.

[48] F. Bullo, J. Cortés, and S. Martínez, *Distributed Control of Robotic Networks*. Princeton, NJ, USA: Princeton University Press, 2009.

[49] Z. Li and Z. Duan, *Cooperative Control of Multi-Agent Systems: A Consensus Region Approach*. Boca Raton, FL, USA: CRC Press, 2014.

[50] R. Bouffanais, *Design and Control of Swarm Dynamics* (SpringerBriefs in Complexity). Singapore: Springer, 2016.

[51] F. F. Lizzio, E. Capello, and G. Guglieri, "A review of consensus-based multi-agent UAV implementations," *Journal of Intelligent & Robotic Systems*, vol. 106, 2022, Art. no. 43.

[52] W. Wang, P. Bai, H. Li, and X. Liang, "Optimal configuration and path planning for UAV swarms using a novel localization approach," *Applied Sciences*, vol. 8, no. 6, 2018, Art. no. 1001.

[53] Y. Sun, "Autonomous integrity monitoring for relative navigation of multiple unmanned aerial vehicles," *Remote Sensing*, vol. 13, 2021, Art. no. 1483.

[54] D. Nemec, A. Janota, M. Hruboš, M. Gregor, and R. Pirník, "Mutual acoustic identification in the swarm of e-puck robots," *International Journal of Advanced Robotic Systems*, 2017.

[55] B. K. Panigrahi, Y. Shi, and M.-H. Lim, Eds., *Handbook of Swarm Intelligence: Concepts, Principles and Applications* (Adaptation, Learning, and Optimization). Berlin, Germany: Springer, 2011, vol. 8.

[56] P. Garcia-Aunon and A. Barrientos Cruz, "Comparison of heuristic algorithms in discrete search and surveillance tasks using aerial swarms," *Applied Sciences*, vol. 8, no. 5, 2018, Art. no. 711.

[57] D. Ziouzios, N. Baras, M. Dasygenis, V. Karayannis, and C. Tsanaktsidis, "Energy efficiency optimization in swarm robotics for smart photovoltaic monitoring," *Energies*, vol. 18, 2025, Art. no. 1587.

[58] A. Kushleyev, D. Mellinger, C. Powers, and V. Kumar, "Towards a swarm of agile micro quadrotors," *Autonomous Robots*, vol. 35, pp. 287–300, 2013.

[59] G. Yi, Z. Feng, T. Mei, P. Li, W. Jin, and S. Chen, "Multi-AGVs path planning based on improved ant colony algorithm," *The Journal of Supercomputing*, vol. 75, pp. 5898–5913, 2019.

[60] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996.

[61] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.

[62] W. J. Gutjahr, "Mathematical runtime analysis of ACO algorithms: Survey on an emerging issue," *Swarm Intelligence*, vol. 1, no. 1, pp. 59–79, 2007.

[63] Y. Chen, S. Ren, Z. Chen, M. Chen, and H. Wu, "Path planning for vehicle-borne system consisting of multi air–ground robots," *Robotica*, vol. 38, pp. 493–511, 2019.

[64] R. Poli, J. Kennedy, and T. Blackwell, "Particle swarm optimization: An overview," *Swarm Intelligence*, vol. 1, no. 1, pp. 33–57, 2007.

[65] W. He, X. Qi, and L. Liu, "A novel hybrid particle swarm optimization for multi-UAV cooperate path planning," *Applied Intelligence*, vol. 51, pp. 7350–7364, 2021.

[66] K.-B. Lee, Y.-J. Kim, and Y.-D. Hong, "Real-time swarm search method for real-world quadcopter drones," *Applied Sciences*, vol. 8, no. 7, 2018, Art. no. 1169.

[67] J. H. Liang and C. H. Lee, "Efficient collision-free path-planning of multiple mobile robots' system using efficient artificial bee colony algorithm," *Advances in Engineering Software*, vol. 79, pp. 47–56, 2015.

[68] D. Karaboga, "An idea based on honey bee swarm for numerical optimization," Erciyes University, Engineering Faculty, Computer Engineering Department, Tech. Rep. TR06, 2005.

[69] X.-S. Yang, "Firefly algorithms for multimodal optimization," in *Stochastic Algorithms: Foundations and Applications (SAGA 2009)*, ser. Lecture Notes in Computer Science, vol. 5792, Springer, 2009, pp. 169–178.

[70] X.-S. Yang, "Flower pollination algorithm for global optimization," in *Unconventional Computation and Natural Computation (UCNC 2012)*, ser. Lecture Notes in Computer Science, vol. 7445, Springer, 2012, pp. 240–249.

[71] L. Cheng, X.-h. Wu, and Y. Wang, "Artificial flora (AF) optimization algorithm," *Applied Sciences*, vol. 8, no. 3, 2018, Art. no. 329.

[72] Y. Tan and Y. Zhu, "Fireworks algorithm for optimization," in *Advances in Swarm Intelligence (ICSI 2010)*, ser. Lecture Notes in Computer Science, vol. 6145, Springer, 2010, pp. 355–364.

[73] S. Mirjalili, "Dragonfly algorithm: A new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems," *Neural Computing and Applications*, vol. 27, pp. 1053–1073, 2016.

[74] K. N. Krishnanand and D. Ghose, "Glowworm swarm optimization for simultaneous capture of multiple local optima of multimodal functions," *Swarm Intelligence*, vol. 3, no. 2, pp. 87–124, 2009.

[75] H. Wang, Y. Li, T. Chang, S. Chang, and Y. Fan, "Event-driven sensor deployment in an underwater environment using a distributed hybrid fish swarm optimization algorithm," *Applied Sciences*, vol. 8, no. 9, 2018, Art. no. 1638.

[76] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.

[77] A. A. Neto, D. G. Macharet, and M. F. M. Campos, "Multi-agent rapidly-exploring pseudo-random tree," *Journal of Intelligent & Robotic Systems*, vol. 89, pp. 69–85, 2017.

[78] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Computer Science Department, Iowa State University, Tech. Rep. TR 98-11, 1998.

[79] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in *Proceedings of the 2000 IEEE International Conference on Robotics and Automation*, 2000, pp. 995–1001.

[80] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[81] Z. Yuan, Z. Yang, L. Lv, and Y. Shi, "A bi-level path planning algorithm for multi-AGV routing problem," *Electronics*, vol. 9, p. 1351, 2020.

[82] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[83] H. Choset, "Coverage of known spaces: The boustrophedon cellular decomposition," *Autonomous Robots*, vol. 9, pp. 247–253, 2000.

[84] E. U. Acar, H. Choset, and J. Y. Lee, "Sensor-based coverage with extended range detectors," *IEEE Transactions on Robotics*, vol. 22, no. 1, pp. 189–198, 2006.

[85] A. Breitenmoser, M. Schwager, J.-C. Metzger, R. Siegwart, and D. Rus, "Voronoi coverage of non-convex environments with a group of networked robots," in *2010 IEEE International Conference on Robotics and Automation*, Anchorage, AK, USA, 2010.

[86] M. Nazarahari, E. Khanmirza, and S. Doostie, "Multi-objective multi-robot path planning in continuous environment using an enhanced genetic algorithm," *Expert Systems with Applications*, vol. 115, pp. 106–120, 2019.

[87] X. Wang, A. Sahin, and S. Bhattacharya, *Coordination-free multi-robot path planning for congestion reduction using topological reasoning*, arXiv, 2022.

[88] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.

[89] B. Wang, K. Zhou, and J. Qu, "Research on multi-robot local path planning based on improved artificial potential field method," in *Proceedings of the Fifth Euro-China Conference on Intelligent Data Analysis and Applications*, Xian, China, 2018, pp. 684–690.

[90] T. Zhao, H. Li, and S. Dian, "Multi-robot path planning based on improved artificial potential field and fuzzy inference system," *Journal of Intelligent & Fuzzy Systems*, vol. 39, pp. 7621–7637, 2020.

[91] C. He, Y. Wan, Y. Gu, and F. L. Lewis, "Integral reinforcement learning-based multi-robot minimum time-energy path planning subject to collision avoidance and unknown environmental disturbances," *IEEE Control Systems Letters*, vol. 5, pp. 983–988, 2021.

[92] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems 30*, 2017.

[93] J. Xue, X. Kong, B. Dong, and M. Xu, *Multi-agent path planning based on MPC and DDPG*, arXiv, 2021.

[94] Z. Zhang, H. Fu, J. Yang, and Y. Lin, "Deep reinforcement learning for path planning of autonomous mobile robots in complicated environments," *Complex & Intelligent Systems*, vol. 11, 2025, Art. no. 277.

[95] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[96] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proceedings of the 35th International Conference on Machine Learning*, 2018, pp. 1587–1596.

[97] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, 2018, pp. 1861–1870.

[98] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *International Conference on Learning Representations (ICLR)*, 2016.

[99] C. Guo, P. Zhu, Z. Zhou, L. Lang, Z. Zeng, and H. Lu, "Imitation learning with graph neural networks for improving swarm robustness under restricted communications," *Applied Sciences*, vol. 11, no. 19, 2021, Art. no. 9055.

[100] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627–635.

[101] D. Oh et al., "A survey on collision avoidance algorithms for multi-robot systems," *International Journal of Control, Automation, and Systems*, vol. 23, no. 7, pp. 2019–2035, 2025.

[102] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research: The 14th International Symposium ISRR*, ser. Springer Tracts in Advanced Robotics, vol. 70, Springer, 2011, pp. 3–19.

[103] D. Zhou, Z. Wang, S. Bandyopadhyay, and M. Schwager, "Fast, on-line collision avoidance for dynamic vehicles using buffered Voronoi cells," *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 1047–1054, 2017.

[104] A. Stentz, "Optimal and efficient path planning for partially-known environments," in *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*, 1994, pp. 3310–3317.

[105] S. Koenig and M. Likhachev, "D* lite," in *Proceedings of the Eighteenth National Conference on Artificial Intelligence (AAAI)*, 2002, pp. 476–483.

[106] D. M. Hart and P. A. Craig-Hart, "Reducing swarming theory to practice for UAV control," in *2004 IEEE Aerospace Conference Proceedings*, Big Sky, MT, USA, 2004.

[107] C. Pinciroli et al., "ARGoS: A modular, parallel, multi-engine simulator for multi-robot systems," *Swarm Intelligence*, vol. 6, no. 4, pp. 271–295, 2012.

[108] M. Rubenstein, A. Cornejo, and R. Nagpal, "Programmable self-assembly in a thousand-robot swarm," *Science*, vol. 345, no. 6198, pp. 795–799, 2014.

[109] M. Yim et al., "Modular self-reconfigurable robot systems: Challenges and opportunities for the future," *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 43–52, 2007.

[110] P. Mahato, S. Saha, C. Sarkar, and M. Shaghil, "Consensus-based fast and energy-efficient multi-robot task allocation," *Robotics and Autonomous Systems*, vol. 159, 2023, Art. no. 104270.

[111] R. Groß, E. Tuci, M. Dorigo, M. Bonani, and F. Mondada, "Object transport by modular robots that self-assemble," in *Proceedings of the 2006 IEEE International Conference on Robotics and Automation*, Orlando, FL, USA, 2006.

[112] M. Kegeleirs and M. Birattari, "Towards applied swarm robotics: Current limitations and enablers," *Frontiers in Robotics and AI*, vol. 12, 2025, Art. no. 1607978.

[113] D. K. Muhsen, A. T. Sadiq, and F. A. Raheem, "A survey on swarm robotics for area coverage problem," *Algorithms*, vol. 17, 2024, Art. no. 3.

[114] M. Yim, K. Roufas, D. Duff, Y. Zhang, C. Eldershaw, and S. Homans, "Modular reconfigurable robots in space applications," *Autonomous Robots*, vol. 14, pp. 225–237, 2003.

[115] J. H. Reif and H. Wang, "Social potential fields: A distributed behavioral control for autonomous robots," *Robotics and Autonomous Systems*, vol. 27, pp. 171–194, 1999.

[116] L. Chen, Y. Shan, W. Tian, B. Li, and D. Cao, "A fast and efficient double-tree RRT*-like sampling-based planner for mobile robotic systems," *IEEE/ASME Transactions on Mechatronics*, vol. 23, no. 6, pp. 2568–2578, 2018.